

**Design, Implementation and Evaluation of a
Reproducible Self-Healing Micro-Batch ETL Pipeline for
Smart City Traffic Data**

by

Birhane Asefa Gebreawahid

a thesis for conferral of a Master of Science in Data Engineering

Dr. Hamza Salem

Name and title of first reviewer

Dr. Fangning Hu

Name and title of second reviewer

Date of Submission: Aug 19, 2026

Statutory Declaration

Family Name, Given/First Name	Gebrewahid, Birhane Asefa
Matriculation number	30008021
What kind of thesis are you submitting: Bachelor-, Master- or PhD-Thesis	Master

English: Declaration of Authorship

I hereby declare that the thesis submitted was created and written solely by myself without any external support. Any sources, direct or indirect, are marked as such. I am aware of the fact that the contents of the thesis in digital form may be revised with regard to usage of unauthorized aid as well as whether the whole or parts of it may be identified as plagiarism. I do agree my work to be entered into a database for it to be compared with existing sources, where it will remain in order to enable further comparisons with future theses. This does not grant any rights of reproduction and usage, however.

This document was neither presented to any other examination board nor has it been published.

German: Erklärung der Autorenschaft (Urheberschaft)

Ich erkläre hiermit, dass die vorliegende Arbeit ohne fremde Hilfe ausschließlich von mir erstellt und geschrieben worden ist. Jedwede verwendeten Quellen, direkter oder indirekter Art, sind als solche kenntlich gemacht worden. Mir ist die Tatsache bewusst, dass der Inhalt der Thesis in digitaler Form geprüft werden kann im Hinblick darauf, ob es sich ganz oder in Teilen um ein Plagiat handelt. Ich bin damit einverstanden, dass meine Arbeit in einer Datenbank eingegeben werden kann, um mit bereits bestehenden Quellen verglichen zu werden und dort auch verbleibt, um mit zukünftigen Arbeiten verglichen werden zu können. Dies berechtigt jedoch nicht zur Verwendung oder Vervielfältigung.

Diese Arbeit wurde noch keiner anderen Prüfungsbehörde vorgelegt noch wurde sie bisher veröffentlicht.

Aug 19,2026



Date, Signature

Acknowledgments

I would like to express my sincere gratitude to everyone who supported me throughout the development of this thesis.

I am particularly grateful to Dr. Hamza Salem for his technical guidance, valuable feedback, and support throughout the project. I would also like to thank Dr. Fangning Hu for her support in reviewing my paper, and assistance in organizing the thesis defense presentation. I am also grateful to Prof. Dr. Stefan Kettemann for his academic supervision and support during the completion of my master's studies.

I would also like to thank my family, friends, and colleagues for their encouragement and support throughout my studies. Finally, I thank Constructor University for providing the academic environment and resources that enabled me to complete this research.

Abstract

The increasing availability of urban traffic data provides new opportunities for improving transportation management in smart cities. However, developing reliable traffic data platforms remains challenging due to reproducible deployment, continuous data processing, workflow reliability, and the reuse of processing solutions across different geographical areas. This thesis addresses these challenges by designing and evaluating a reproducible self-healing micro-batch Extract Transform Load (ETL) pipeline for smart city traffic data analytics.

The proposed architecture integrates automated workflow orchestration, distributed data processing, standardized data management, analytical storage, and interactive visualization. Traffic flow and incident data are collected from the TomTom Traffic API and processed using Apache Airflow and Apache Spark. Configuration-driven processing and schema standardization enable the same workflow to support multiple cities. Docker Compose and Terraform are used to support reproducible deployment and infrastructure automation.

The implemented system is evaluated based on functional correctness, performance, reproducibility, resilience, and reusability. The evaluation examines data processing execution, workflow reliability, deployment consistency, monitoring, and the extension of the pipeline to different city configurations. The results demonstrate that the architecture successfully automates traffic data processing, supports workflow recovery and monitoring, and enables reusable multi-city processing.

The main contributions are a reproducible data engineering architecture for smart city traffic analytics, a self-healing micro-batch processing workflow, and a configuration-based approach for reusable multi-city processing. The developed framework provides a practical foundation for maintainable traffic data platforms and future extensions toward large-scale streaming and advanced analytics.

Keywords: Smart Cities, Data processing, ETL Pipeline, Apache Airflow, Apache Spark, Micro-Batch Processing, Data Engineering, Reproducibility, Self-Healing Systems, Infrastructure as Code, TomTom API

List of Abbreviations

AMI	Amazon Machine Image.
AWS	Amazon Web Services.
DAG	Directed Acyclic Graph.
EC2	Elastic Compute Cloud.
ETL	Extract Transform Load.
GPS	Global Positioning System.
IaC	Infrastructure as Code.
IoT	Internet of Things.
ITS	Intelligent Transportation Systems.
JSONL	JavaScript Object Notation Lines.
KPI	Key Performance Indicator.
SQL	Structured Query Language.
SSH	Secure Shell.

Contents

Acknowledgments	iii
List of Abbreviations	i
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Research Objectives	3
1.4 Research Questions	3
1.5 Research Methodology Overview	3
1.6 Scope and Limitations	4
1.7 Thesis Organization	5
2 Literature Review	6
2.1 Smart City Concepts and Traffic Management	6
2.2 Real-Time vs. Batch vs. Micro-Batch Data Processing	7
2.3 ETL Pipeline Architectures in Urban Data Systems	8
2.4 Medallion Architecture for Data Engineering Systems	10
2.5 Workflow Orchestration and Self-Healing Data Pipelines	11
2.6 Technologies for Scalable and Reproducible Data Processing	11
2.7 Related Work and Gap Analysis	12
3 System Design and Architecture	14
3.1 Architecture Overview and Design Principles	14
3.2 Data Source and Multi-City Design	16
3.3 Data Processing Architecture	16
3.4 Workflow Orchestration Design	17
3.5 Data Storage and Analytical Architecture	18
3.6 Deployment Architecture	19
3.7 Overall System Workflow	19
4 System Implementation	20
4.1 Implementation Overview	20

4.2	Data Acquisition and API Integration	20
4.3	Workflow Orchestration	22
4.4	Distributed Data Processing	24
4.4.1	Medallion data processing architecture	24
4.4.2	Data Cleaning and Validation	25
4.4.3	Feature Engineering	25
4.4.4	Data Partitioning Strategy	27
4.5	Multi-City Data Processing Implementation	27
4.5.1	City Configuration and Grid Generation	27
4.6	Data Warehouse, Storage, and Monitoring	28
4.7	Data Visualization Dashboards	29
4.8	Alerting Implementation	32
4.9	Infrastructure and Container Deployment	34
4.9.1	Source Code Repository	36
4.9.2	Implementation Challenges and Resolutions	36
5	Evaluation and Discussion	37
5.1	Evaluation Strategy	37
5.1.1	Functional and Unit Testing	38
5.1.2	Performance Benchmarking	38
5.1.3	Integration Testing	39
5.2	Reproducibility Evaluation	40
5.3	Pipeline Self Healing Evaluation	41
5.4	Multi-City Reusability Evaluation	41
5.5	Data Processing Performance	42
5.6	Dashboard and Alerting Evaluation	43
5.7	Discussion	44
5.8	Summary of Findings	44
6	Conclusion and Future Work	45
6.1	Research Summary	45
6.2	Research Contributions	45
6.3	Research Questions Revisited	46
6.4	Limitations	47
6.5	Future Work	47

List of Figures

1.1	Research methodology overview	4
3.1	High-level architecture of the proposed smart city traffic ETL platform .	15
3.2	Spark Medallion data layer design for the data processing	17
3.3	Airflow workflow orchestration design	18
3.4	pipeline execution flow	19
4.1	Sample raw traffic data collected in JSONL format	22
4.2	Overview of the five implemented Airflow DAGs	22
4.3	Airflow orchestration for traffic flow pipeline	23
4.4	Airflow orchestration for incidents pipeline	23
4.5	Data partition based on date and city	27
4.6	Configuration structure used for multi-city traffic data collection	28
4.7	Overall architecture of the implemented pipeline with data storage, ana- lytical warehouse, and monitoring components.	29
4.8	Traffic flow visualization dashboards	30
4.9	Traffic incidents visualization dashboards	31
4.10	Pipeline evaluation and monitoring dashboards	32
4.11	Incident alert email notification	33
4.12	Startup of the Docker Compose environment	34
4.13	Local vs Terraform-based deployment of the Dockerized ETL environment	34
5.1	Execution of the automated test suite	38
5.2	City-grid generation benchmark for Berlin, Bremen, and Frankfurt	39
5.3	Successful execution of the city configuration integration test	40
5.4	Performance overview of the pipeline	43

List of Tables

2.1	Comparison of Batch, Micro-Batch, and Streaming Data Processing . . .	8
2.2	Evolution of ETL Pipeline Architectures	9
2.3	Characteristics of Medallion Architecture Layers	10
2.4	Summary of Related Work and Research Gaps	12
4.1	Raw traffic flow variables collected during data acquisition	21
4.2	Raw incident variables collected during data acquisition	21
4.3	Airflow DAG responsibilities and trigger relationships	24
4.4	Traffic flow feature engineering variables	26
4.5	Traffic incident feature engineering variables	26
4.6	Infrastructure provisioning time comparison	35
5.1	Functional and unit test results	38
5.2	City-grid generation performance	39
5.3	Integration test result	40
5.4	Airflow DAG resilience evaluation	41
5.5	Multi-city processing evaluation	41
5.6	Summary of evaluation results	44

Chapter 1

Introduction

1.1 Background and Motivation

Smart city initiatives increasingly use digital technologies to collect, process, and analyse urban data to support decision making [1, 2]. Transportation is an important application area because continuous data collection can provide information about traffic conditions, road incidents, and mobility patterns for traffic monitoring, operational planning, and intelligent transportation services [3].

The availability of traffic data, however, does not by itself provide data that can be directly used for analysis. Traffic information is collected from heterogeneous sources with different formats, structures, and update frequencies. Data engineering pipelines are therefore required to ingest raw information, transform it into structured datasets, store the processed data, and make it available for analytical applications. In a continuously operating environment, these pipelines must also account for failures in external data sources, processing stages, and infrastructure.

A further requirement arises when the same processing workflow is intended to support more than one geographical area. Traffic processing systems may depend on city-specific geographical parameters and data characteristics. If these parameters are embedded directly in processing logic, extending a pipeline to another city can require duplicated implementations and additional maintenance. Separating city-specific configuration from common processing logic and using standardized data structures can instead provide a basis for reusable multi-city processing.

These requirements motivate the combination of three data-engineering properties in this thesis: reproducible deployment, resilient workflow execution, and reusable multi-city processing. Reproducibility concerns the ability to recreate the pipeline environment consistently; resilience concerns the workflow’s ability to detect processing failures and recover automatically, while reusability concerns the application of the same processing workflow across different cities through configuration and standardized schemas.

Based on these requirements, this thesis proposes and evaluates a reproducible self-healing micro-batch ETL pipeline for smart city traffic data. The pipeline collects traffic flow and incident information from the TomTom Traffic API and processes the data using Apache Airflow and Apache Spark. Multiple cities are supported through configuration-based processing and standardized schemas. Docker Compose and Terraform are used for deployment and infrastructure management, while processed data is made available for analytical use through the storage and visualization components of the system.

1.2 Problem Statement

A continuously operating traffic ETL pipeline must maintain consistent deployment, handle failures during data processing, and support the reuse of common processing logic across different cities. These requirements introduce three specific engineering problems.

First, reproducing a data engineering environment can be difficult when the system depends on multiple software components, infrastructure resources, and runtime dependencies. Without automated deployment procedures and standardized environments, recreating the same pipeline in another environment can require manual configuration and may produce inconsistent results [4].

Second, continuous traffic processing depends on external data sources and multiple workflow stages. Network interruptions, unavailable services, and processing errors can interrupt individual tasks or entire workflow runs. Without monitoring and recovery mechanisms, these failures can interrupt continuous processing and reduce the availability of processed traffic data.

Third, applying a traffic processing workflow to multiple cities requires handling geographical and configuration differences without duplicating the common processing logic. City-specific parameters, such as geographical configurations, should therefore be separated from the common workflow, while the resulting data should follow standardized schemas that allow the processed information to be handled consistently.

Existing research addresses areas including smart city analytics, traffic monitoring, and distributed data processing. However, the combination of reproducible deployment, resilient workflow execution, and reusable multi-city processing within a single practical data engineering architecture remains the focus of this research.

The problem addressed by this thesis is the design and evaluation of a micro-batch data engineering architecture for continuous smart city traffic data processing that supports reproducible deployment, automatic workflow recovery, and reuse across different cities.

1.3 Research Objectives

The research focuses on four key objectives: first, to design a reproducible data engineering architecture using containerization and infrastructure automation to ensure consistent deployment and environment configuration; second, to implement a self-healing micro-batch ETL workflow with automated orchestration, monitoring, logging, and recovery mechanisms for recurring traffic data processing; third, to develop a reusable multi-city traffic data processing framework based on configuration-driven parameters and standardized schemas, enabling the same ETL workflow to support different geographical locations; and finally, to evaluate the developed system in terms of functional correctness, processing performance, reproducibility, resilience, and reusability.

1.4 Research Questions

Based on the identified problem and research objectives, this thesis investigates the following research questions:

- **RQ1: Reproducibility** How can an ETL pipeline for smart-city traffic data achieve reproducible deployment and execution across different environments?
- **RQ2: Resilience** How can an ETL pipeline for smart-city traffic data detect and recover from failures during recurring data ingestion and processing with minimal manual intervention?
- **RQ3: Reusability** How can an ETL pipeline for smart-city traffic data support processing across multiple cities through configuration-driven processing and standardized output schemas?

1.5 Research Methodology Overview

This research follows a design and evaluation methodology for developing and assessing a practical data engineering solution for smart city traffic analytics. The research consists of three main phases: system design, implementation, and evaluation.

The first phase analyses the requirements derived from the identified problem and defines an architecture for continuous traffic data collection, distributed processing, analytical storage, and visualization. The design separates common processing logic from city-specific configuration and incorporates mechanisms for workflow monitoring and recovery.

The second phase implements the designed architecture as a micro-batch ETL pipeline. The implementation integrates automated workflow orchestration, distributed data processing, standardized data management, monitoring, and reproducible deployment components. Traffic flow and incident data from multiple cities are processed using the same general workflow.

The final phase evaluates the implemented system according to the defined evaluation criteria. Functional correctness, processing performance, deployment reproducibility, workflow resilience, and multi-city reusability are examined using the evaluation procedures described in Chapter 5.

The overall research methodology is summarized in Figure 1.1.

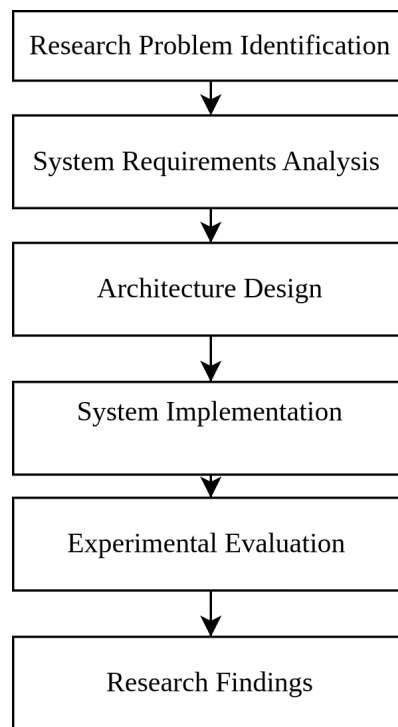


Figure 1.1: Research methodology overview

1.6 Scope and Limitations

This research focuses on the design, implementation, and evaluation of a data engineering pipeline for smart city traffic data processing. The scope covers the collection, transformation, storage, monitoring, and visualization of traffic flow and incident data using a reproducible micro-batch ETL architecture.

The pipeline uses the TomTom Traffic API as its traffic data source and supports multiple cities through configuration-based processing and standardized data schemas. The technical scope includes workflow orchestration, distributed processing, analytical storage, monitoring, and reproducible deployment.

The research does not address advanced traffic analytics applications such as traffic prediction, route optimization, or machine-learning-based forecasting. It also focuses on micro-batch processing rather than fully streaming architectures.

1.7 Thesis Organization

The thesis is organized into six chapters. **Chapter 1: Introduction** presents motivation, problem statement, research objective, research questions, and scope. **Chapter 2: Literature Review** reviews existing research on smart city traffic analytics, ETL architectures, and related technologies, and identifies the research gap addressed by this thesis. **Chapter 3: System Design and Architecture** presents the proposed architecture, including data sources, the Medallion design, orchestration, storage, and deployment. **Chapter 4: System Implementation** describes the implementation of the proposed architecture, covering ingestion, processing, multi-city support, monitoring, dashboards, and deployment. **Chapter 5: Evaluation and Discussion** evaluates the system against functional, performance, reproducibility, resilience, and reusability criteria, and discusses the findings and limitations. **Chapter 6: Conclusion and Future Work** summarizes the contributions, revisits research questions, and discusses future work.

Chapter 2

Literature Review

2.1 Smart City Concepts and Traffic Management

Smart cities integrate information and communication technologies, Internet of Things (IoT) devices, cloud computing, and data analytics to collect and process data from urban environments and support data-driven services. Within this context, Intelligent Transportation Systems (ITS) are an important application area, as they use continuously collected traffic data to monitor road conditions, congestion, and traffic incidents and to support traffic management decisions [5].

The deployment of Global Positioning System (GPS)-enabled vehicles, road sensors, mobile devices, and connected navigation services has increased the availability of traffic data. These sources generate heterogeneous observations, including vehicle speed, travel time, congestion level, and traffic incidents. Transforming such raw data into consistent analytical datasets requires data engineering pipelines that support repeated ingestion, validation, transformation, storage, and downstream analysis [6, 7].

The increasing volume and frequency of traffic observations have also created a need for efficient data processing approaches. Distributed processing frameworks such as Apache Spark enable parallel processing of large datasets and are widely used for data-intensive analytics [8]. However, processing performance alone does not address the operational requirements of continuously running traffic ETL workflows. Such pipelines must also provide reliable execution, recover from transient failures, and support consistent processing when applied to different geographical areas.

Traditional batch processing is suitable for periodic and historical analysis, but traffic monitoring can require more frequent data processing to reduce the delay between data collection and analysis. This has motivated the use of streaming and micro-batch processing approaches. Micro-batch processing provides a practical approach for repeated traffic data ingestion by processing data in small, scheduled batches while retaining the manageability of batch-oriented workflows.

2.2 Real-Time vs. Batch vs. Micro-Batch Data Processing

The increasing availability of continuously generated data has influenced the development of modern data processing architectures. Data processing systems can broadly be categorized according to how they handle incoming data, including batch processing, continuous stream processing, and micro-batch processing. These approaches differ in their processing latency, throughput, resource requirements, and fault-tolerance mechanisms. In particular, micro-batch processing provides a compromise between the high throughput of batch processing and the low latency associated with continuous stream processing by processing incoming data in small groups at short intervals [9, 10]. Selecting an appropriate processing model is therefore an important architectural consideration for data-intensive applications, particularly in smart city traffic management, where data are generated continuously but operational requirements may range from periodic analytics to near-real-time responses.

Batch processing is the traditional approach for executing data transformation tasks. In this model, data are accumulated over a predefined period before processing begins. Batch processing is highly suitable for historical analysis, reporting, and large-scale analytical workloads because it efficiently processes substantial volumes of data with relatively simple execution models. Kimball and Ross emphasize that traditional data warehouse architectures commonly rely on scheduled batch ETL processes to populate analytical databases [11]. Similarly, Reis and Housley describe batch processing as an effective solution for applications where immediate data availability is not required and computational efficiency is prioritized [7].

In contrast, real-time streaming systems process data immediately as events are generated, enabling very low processing latency. Streaming architectures are widely adopted in applications requiring continuous monitoring, fraud detection, financial transactions, and industrial automation. However, maintaining continuous processing introduces additional challenges, including state management, fault recovery, event ordering, and operational complexity. These systems often require specialized messaging infrastructures and sophisticated stream-processing frameworks to guarantee reliable event delivery and processing consistency [7].

Micro-batch processing represents a compromise between traditional batch processing and true streaming. Instead of processing individual events continuously, incoming data are grouped into small batches collected over short time intervals before processing. This approach substantially reduces processing latency compared with conventional batch systems while preserving many of the operational advantages associated with batch execution, including simplified scheduling, deterministic processing, and improved fault

recovery. Apache Spark supports micro-batch processing through its distributed execution model, enabling scalable and fault-tolerant processing of continuously arriving datasets [8].

For many smart city traffic applications, micro-batch processing provides a practical balance between timeliness and implementation complexity. Traffic conditions generally evolve over minutes rather than milliseconds, making extremely low-latency streaming architectures unnecessary for many analytical and operational tasks. Processing traffic observations at regular intervals enables efficient aggregation, feature engineering, data validation, and storage while maintaining sufficiently up-to-date information for visualization and decision support.

Table 2.1 summarizes the major characteristics of the three processing paradigms discussed in this section.

Table 2.1: Comparison of Batch, Micro-Batch, and Streaming Data Processing

Characteristic	Batch	Micro-Batch	Streaming
Processing latency	High	Medium	Low
Implementation complexity	Low	Medium	High
Fault tolerance	High	High	Medium
Scheduling	Periodic	Frequent intervals	Continuous
Typical applications	Reporting	Near real-time analytics	Event processing
Resource utilization	Efficient	Balanced	Continuous

Considering the characteristics summarized in Table 2.1, micro-batch processing provides an effective balance between processing latency, scalability, and operational simplicity. Consequently, it has become an attractive architectural choice for modern traffic data engineering systems that require frequent data updates without the complexity associated with fully streaming infrastructures.

2.3 ETL Pipeline Architectures in Urban Data Systems

An ETL pipeline consists of three primary stages: extraction, transformation, and loading. During the extraction phase, data are collected from one or more heterogeneous

sources, including databases, application programming interfaces, sensors, and external data providers. The transformation phase applies operations such as data cleaning, validation, normalization, enrichment, and feature engineering to improve data quality and analytical usefulness. Finally, the loading phase stores processed data into analytical repositories such as data warehouses, data lakes, or operational databases for downstream applications [11, 6].

Traditional data warehouse architectures commonly follow structured ETL workflows where operational data are periodically extracted, transformed according to predefined business rules, and loaded into analytical systems. Kimball and Ross describe dimensional modeling and ETL processes as essential components for developing reliable analytical data warehouses that support reporting and decision-making activities [11]. However, the rapid growth of large-scale and continuously generated datasets has introduced new challenges related to scalability, processing speed, and system reliability.

Modern data engineering architectures have therefore evolved from centralized ETL systems toward distributed and cloud-based processing frameworks. According to Reis and Housley, contemporary data platforms require scalable architectures capable of managing diverse data sources, enabling automated workflows, ensuring data quality management, and providing reliable data delivery mechanisms [7]. These requirements are particularly relevant for smart city applications where traffic sensors, connected vehicles, and location-based services continuously generate large volumes of time-dependent data.

Distributed processing frameworks have become an important component of modern ETL architectures because they enable parallel computation across multiple processing nodes. Apache Spark introduced a unified processing engine that supports large-scale data transformation workloads through distributed memory computation and fault-tolerant execution mechanisms [8]. Such frameworks allow ETL pipelines to perform complex operations, including filtering, aggregation, and feature engineering, on datasets that exceed the processing capacity of traditional single-machine systems.

Table 2.2: Evolution of ETL Pipeline Architectures

Architecture	Data Processing	Advantages	Limitations
Traditional ETL	Scheduled batch processing	Simple and reliable	Limited scalability and latency
Distributed ETL	Parallel processing frameworks	Handles large datasets	Higher complexity
Cloud-native ETL	Automated scalable services	Flexible deployment	Requires orchestration and monitoring

Although modern ETL architectures provide improved scalability and processing ca-

pabilities, several operational challenges remain. Data pipelines must ensure reliable execution, recover from failures, maintain consistent environments, and support deployment across different infrastructures. These challenges have motivated the adoption of architectural patterns that improve data quality and reliability, such as layered data architectures and automated workflow orchestration. The following section discusses the Medallion Architecture, which addresses the organization and management of data throughout different processing stages.

2.4 Medallion Architecture for Data Engineering Systems

Modern data engineering systems require structured approaches for managing data throughout different stages of processing. Layered data architectures address this requirement by separating raw data storage, data refinement, and analytical outputs. This separation improves data traceability, maintainability, and the ability to reprocess data when transformation requirements change.

The Medallion Architecture is a layered data design pattern that organizes data into progressively refined stages, typically represented by Bronze, Silver, and Gold layers [12]. The Bronze layer retains raw source data with minimal transformation, preserving the original data for auditing and reprocessing. The Silver layer contains cleaned, validated, and standardized data prepared for further processing and analysis. The Gold layer provides refined, analysis-ready datasets optimized for reporting, visualization, and decision-making. This layered organization establishes clear boundaries between data ingestion, transformation, and analytical consumption. Consequently, it can improve data quality, traceability, and maintainability while facilitating the reuse of intermediate datasets across different analytical workloads. These characteristics make the Medallion Architecture particularly relevant to data-intensive applications in which raw data must be progressively transformed into reliable analytical datasets.

Table 2.3: Characteristics of Medallion Architecture Layers

Layer	Purpose	Main Operations	Output
Bronze	Raw data preservation	Data ingestion, storage, metadata capture	Original datasets
Silver	Data refinement	Cleaning, validation, normalization, feature engineering	Standardized datasets
Gold	Analytical consumption	Aggregation, KPI generation, optimization	Analytics-ready datasets

Although the Medallion Architecture improves data organization and processing reliability, managing multi-stage data pipelines also requires automated execution, scheduling, monitoring, and failure recovery mechanisms. Therefore, the following section discusses workflow orchestration approaches and their role in managing reliable modern data engineering pipelines.

2.5 Workflow Orchestration and Self-Healing Data Pipelines

Modern data pipelines often consist of multiple interdependent tasks that require automated scheduling, dependency management, monitoring, and failure handling. Workflow orchestration addresses these requirements by coordinating processing activities and their dependencies, commonly through Directed Acyclic Graph (DAG)s. Apache Airflow has been widely adopted for DAG-based workflow management, providing mechanisms for scheduling, dependency management, monitoring, and integration with heterogeneous processing environments [13].

Reliable data pipelines also require mechanisms for recovering from operational failures. Common approaches include automatic retries, failure detection, checkpointing, and monitoring. Self-healing pipelines extend these mechanisms by automatically detecting failures and initiating predefined recovery actions, thereby reducing manual intervention and improving pipeline resilience. Such approaches are particularly relevant to continuously operating data processing systems in which temporary failures should not interrupt the complete workflow.

2.6 Technologies for Scalable and Reproducible Data Processing

Scalable data engineering architectures commonly combine distributed processing, workflow orchestration, reproducible deployment, optimized storage, and analytical interfaces. Distributed processing frameworks such as Apache Spark have been extensively used for large-scale data transformation and analytics, providing parallel execution and fault-tolerant processing across clusters [8]. For reproducible deployment, containerization provides a way to package applications and their dependencies into isolated environments, while Infrastructure as Code (IaC) approaches support automated and repeatable infrastructure provisioning [14, 15].

Storage technologies also influence the efficiency of analytical workloads. Columnar formats such as Apache Parquet have been evaluated for large-scale analytical processing and can provide advantages in storage efficiency and query performance [16]. Relational database systems such as PostgreSQL provide structured data management and

Structured Query Language (SQL)-based access and have their origins in the POSTGRES database research project [17]. At the analytical consumption layer, visualization platforms such as Apache Superset have been used and evaluated for interactive data exploration and dashboard-based analytics [18].

Together, these technologies address complementary aspects of data engineering: distributed processing supports scalable computation, orchestration coordinates workflow execution, containerization and IaC support reproducible deployment, and analytical storage and visualization facilitate downstream data consumption. Their combination provides a technological foundation for building scalable and reproducible data processing pipelines.

2.7 Related Work and Gap Analysis

Existing research has addressed different aspects of smart city data processing, including traditional ETL, distributed computation, layered data architectures, and urban data analytics. Traditional ETL approaches provide established methods for data integration and warehousing [11], while Apache Spark supports scalable and fault-tolerant distributed processing [8]. Modern data engineering literature also emphasizes scalable and cloud-oriented architectures [7]. However, these approaches do not fully address reproducible deployment, automated recovery, and pipeline reuse across different urban environments.

Layered architectures such as the Medallion Architecture support progressive data refinement through Bronze, Silver, and Gold layers [12]. Smart city research has also demonstrated the value of integrating heterogeneous urban data for monitoring and decision-making [19]. However, these approaches generally focus on data organization or analytical applications rather than integrating layered processing with reproducible deployment and self-healing workflow execution.

Table 2.4 summarizes the main limitations relevant to this thesis.

Table 2.4: Summary of Related Work and Research Gaps

Source	Main Focus	Gap Relevant to This Thesis
Kimball and Ross (2013) [11]	Traditional ETL and data warehousing	Limited support for continuously generated and distributed workloads.
Zaharia et al. (2016) [8]	Distributed and fault-tolerant processing	Does not address infrastructure reproducibility or workflow-level recovery.
Reis and Housley (2022) [7]	Modern data engineering architectures	Provides broad architectural guidance rather than an integrated multi-city traffic pipeline.
Zanella et al. (2014) [19]	Smart city data and applications	Focuses on urban applications rather than ETL pipeline engineering.

Based on these observations, three gaps are identified: reproducible deployment, automated recovery of failed workflows, and reusable processing across multiple cities. These gaps correspond to RQ1, RQ2, and RQ3, respectively. This thesis addresses them through a micro-batch ETL architecture.

Chapter 3

System Design and Architecture

3.1 Architecture Overview and Design Principles

The required pipeline is designed as a reproducible and reusable micro-batch ETL platform for smart city traffic analytics. The architecture integrates data acquisition, workflow orchestration, distributed processing, storage, and analytical presentation into a modular pipeline.

The architecture consists of six logical components: infrastructure, data acquisition, orchestration, processing, storage, and presentation. The infrastructure component provides the execution environment. The data acquisition component handles communication with external traffic data sources. The orchestration component coordinates dependent processing activities and their execution states. The processing component performs data transformation and refinement, while the storage component maintains intermediate and analytical datasets. The presentation component provides access to processed information through dashboards and notifications.

The design is guided by the three research objectives of reproducibility, resilience, and reusability. Reproducibility is supported through the separation of environment configuration from application logic and the definition of deployment requirements as reusable configuration. Resilience is supported through the decomposition of the pipeline into manageable stages and the incorporation of automated failure recovery. Reusability is supported through configuration-driven processing and common data structures, enabling the same pipeline logic to be applied across multiple cities.

The components are loosely coupled so that changes or failures in one stage have limited impact on other stages. Raw data are retained to support traceability and reprocessing, while downstream processing operates on standardized datasets. Figure 3.1 presents the proposed architecture and the relationships between its main components and data flows.

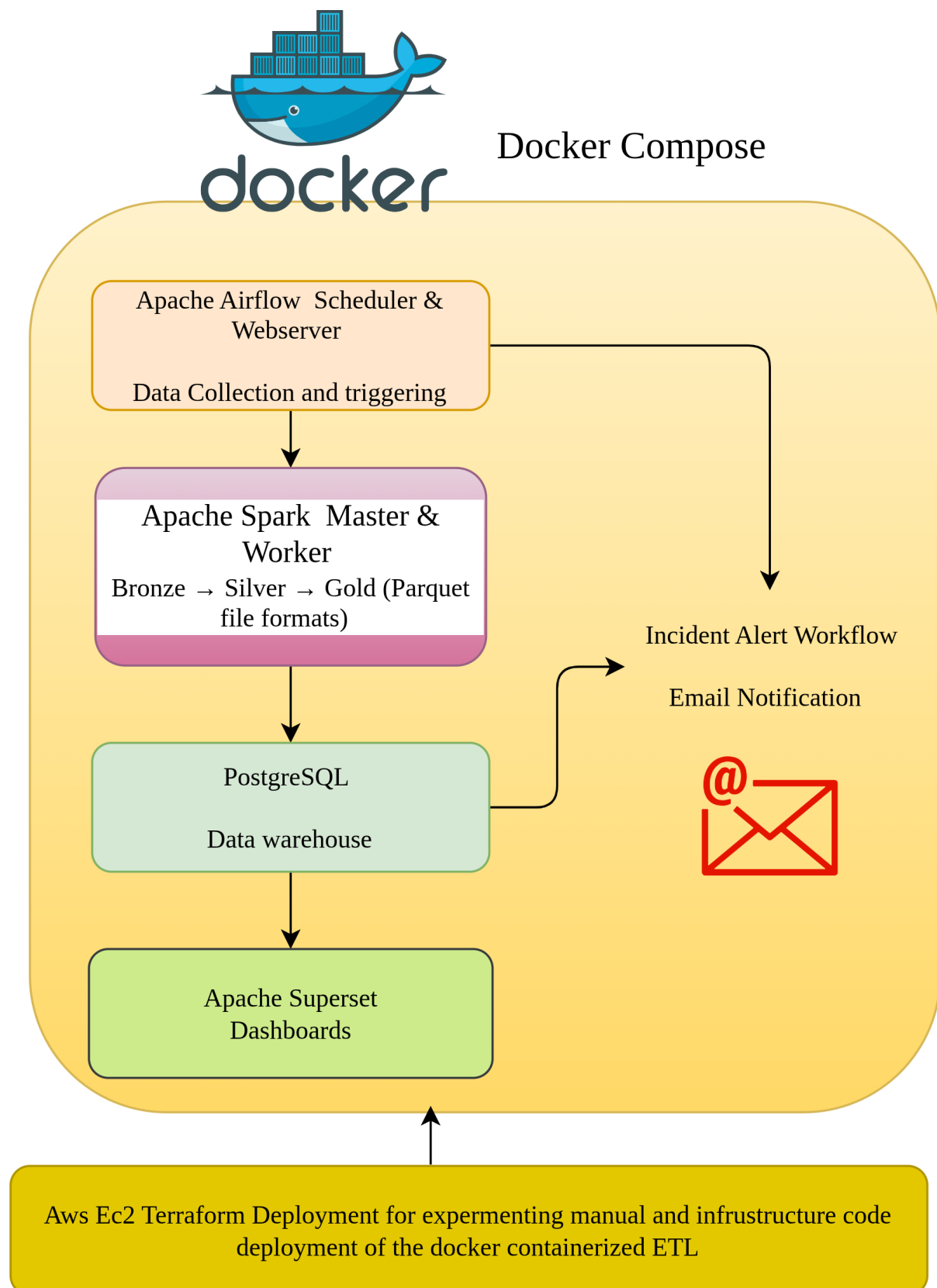


Figure 3.1: High-level architecture of the proposed smart city traffic ETL platform

3.2 Data Source and Multi-City Design

TomTom Traffic API is selected as the real external data source for smart traffic city[20]. Several traffic datas were considered during the source selection process, including commercial mapping and routing services. TomTom was selected because it provides traffic-specific APIs with real-time flow and incident data, and offers a free evaluation tier suitable for academic development and experimentation. The Traffic Flow service provides current and free-flow speeds for individual road segments, while the Incident Details service provides information about current traffic incidents and their associated locations and delays [20]. This combination directly matches the data requirements of the proposed ETL pipeline and avoids relying on a routing-oriented service where traffic information is primarily provided as part of route calculations.

The pipeline therefore consists of two main data streams: a traffic flow pipeline and an incident pipeline. Flow Segment API is chosen to provide traffic flow data, while the Incident Details API is chosen to provide traffic incident information. Both pipelines are designed to process data for multiple selected cities using the same reusable pipeline structure, allowing the system to collect and process traffic flow and incident data consistently across different city configurations. Berlin, Bremen, and Frankfurt are selected to evaluate the multi-city configuration and the reusability of the pipeline across different geographic areas. Geographic coverage is represented using city-specific bounding boxes and grid configurations. The detailed coordinates and grid-generation implementation are presented in Chapter-4.

3.3 Data Processing Architecture

The processing architecture follows Medallion (Bronze, Silver, and Gold) organization as described in Chapter 2 [12]. The three layers separate raw data preservation, data refinement, and analytical consumption.

Apache Spark is selected as the distributed processing framework for the transformation stages because it provides scalable processing of the collected traffic data and supports efficient transformation of structured and semi-structured datasets. Its DataFrame-based processing model also supports the cleaning, standardization, aggregation, and transformation operations required across the processing stages. The Bronze layer is designed to preserve collected data, the Silver layer to provide cleaned and standardized datasets, and the Gold layer to produce consumption-ready analytical outputs.

This separation supports traceability and reprocessing while allowing downstream analytical datasets to be developed independently from the original data collection process. Figure 3.2 illustrates the processing design.

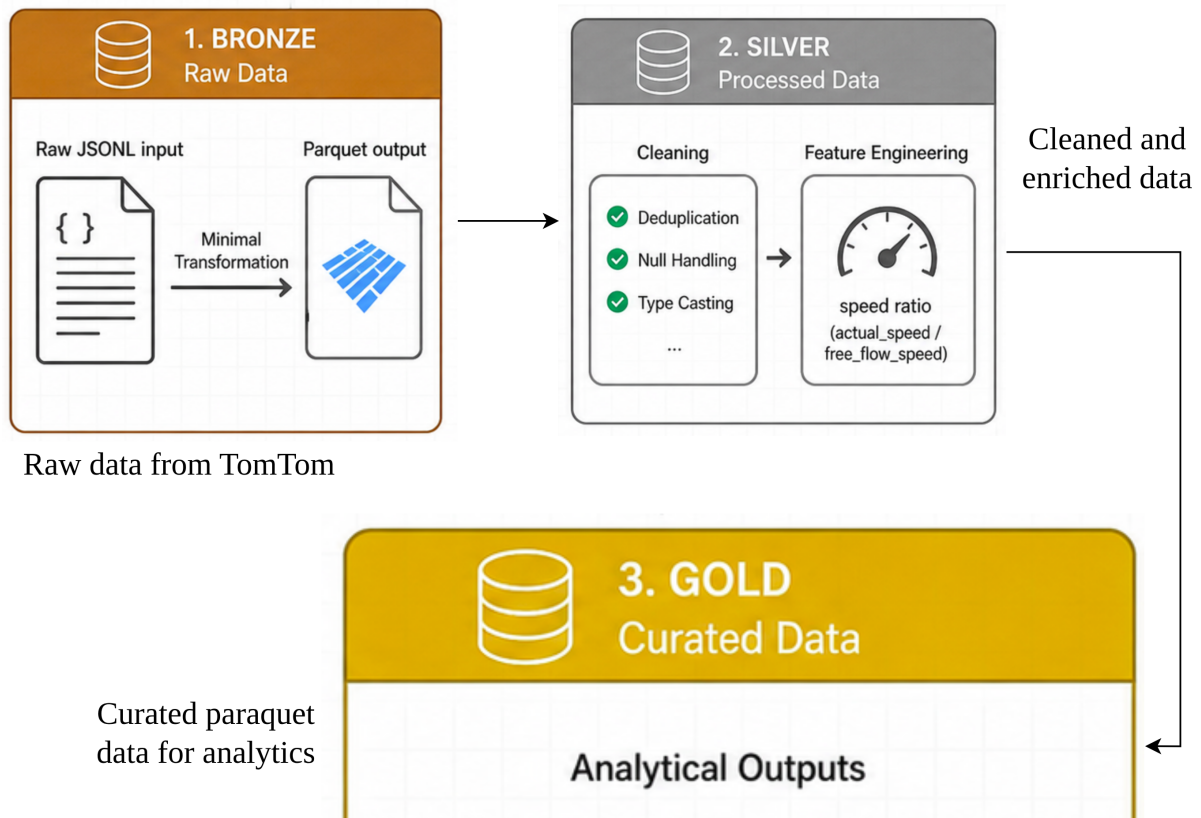


Figure 3.2: Spark Medallion data layer design for the data processing

3.4 Workflow Orchestration Design

Apache Airflow is selected as the workflow orchestration platform [13]. Its DAG-based model provides a suitable structure for representing dependencies between data acquisition, processing, and downstream activities.

Traffic flow and incident processing are designed as separate workflows to provide independent execution and failure isolation. A dedicated alerting path is also included for processed traffic incidents.

The orchestration design supports the resilience objective through controlled task dependencies and automated failure recovery. As shown in Figure 3.3 Apache Airflow handles scheduling, triggering Spark jobs, task dependencies, retry attempts, exponential backoff, failure handling and monitoring.

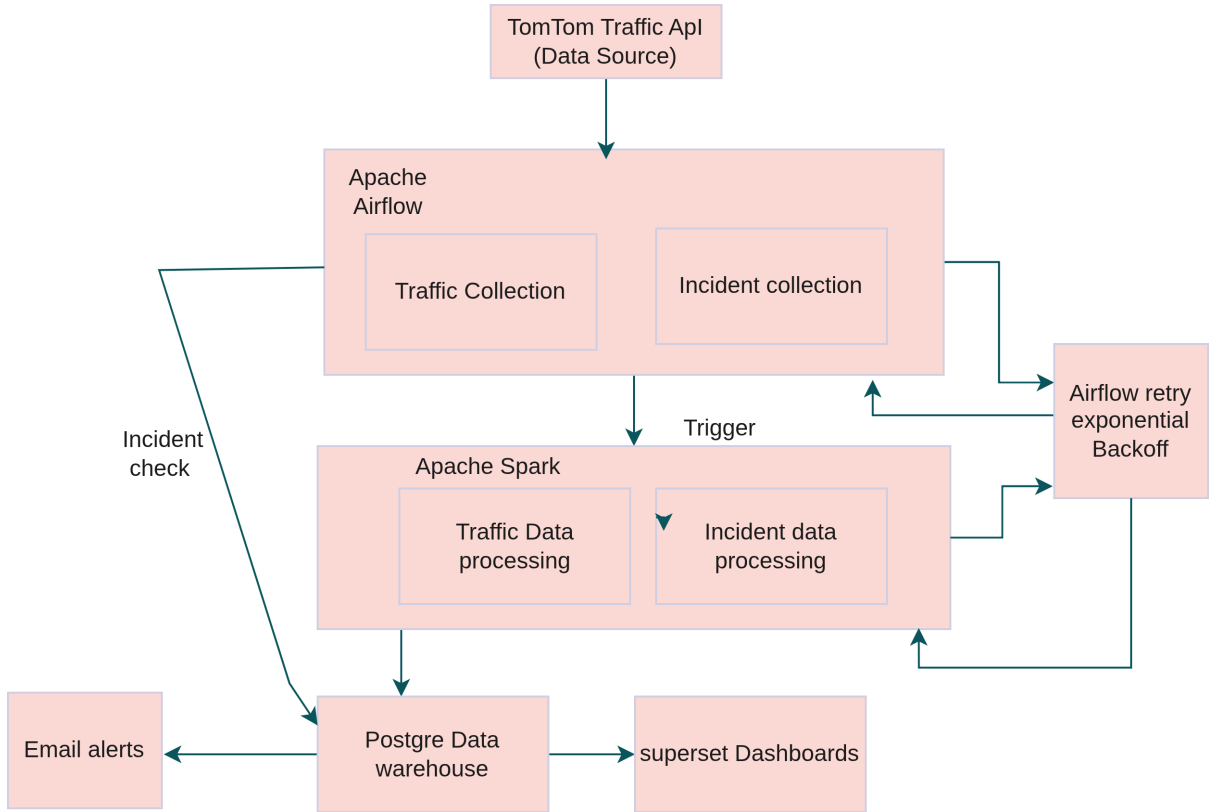


Figure 3.3: Airflow workflow orchestration design

3.5 Data Storage and Analytical Architecture

Parquet is selected as the intermediate data format because it provides a structured representation suitable for distributed analytical processing. PostgreSQL is selected as the relational storage layer for curated analytical and pipeline monitoring data.

The analytical storage is designed to support three main purposes: traffic analysis, incident analysis, and pipeline performance monitoring for evaluation purposes. Apache Superset is selected as the visualization platform for presenting the curated datasets. Superset is suitable for the proposed architecture because it can be deployed using Docker Compose, allowing it to be integrated into the containerized application environment alongside the other pipeline services. It also provides direct connectivity to PostgreSQL and supports SQL-based data exploration through SQL Lab, which is useful for querying and validating the curated datasets. Furthermore, Superset provides configurable dashboards, filters, and a wide range of visualization types, allowing the presentation layer to be adapted to the traffic analytics requirements of the system [21]. Its open-source nature and configuration flexibility also support the reproducibility and maintainability objectives of the proposed deployment

3.6 Deployment Architecture

Terraform[22] is selected for infrastructure provisioning, with Amazon Web Services (AWS) Elastic Compute Cloud (EC2) chosen as the cloud-based execution environment. Docker[23] Compose is chosen to define and manage the containerized application environment deployed on the EC2 instance.

This separation distinguishes infrastructure configuration from application runtime configuration and supports the reproducibility objective of the proposed architecture. Terraform is responsible for provisioning the underlying EC2 infrastructure, while Docker Compose manages the services required to execute the pipeline, including Apache Airflow, Apache Spark, PostgreSQL, and Apache Superset.

The deployment design is intended to support both local and cloud-based execution environments. The same containerized application environment can therefore be executed locally using Docker Compose or deployed on the provisioned AWS EC2 infrastructure.

The specific infrastructure resources, container services, configuration files, and deployment procedures are described in Chapter 4.

3.7 Overall System Workflow

The process follows a staged flow from external traffic data sources to analytical applications. Traffic flow and incident data enter through separate acquisition paths and are processed through their corresponding workflows. The resulting curated datasets are then made available through the analytical storage and presentation layers.

The architecture maintains separation between traffic and incident processing while allowing both paths to share common infrastructure and processing principles. Incident processing additionally connects to the alerting component for event-based notifications.

Figure 3.4 summarizes the overall designed workflow.

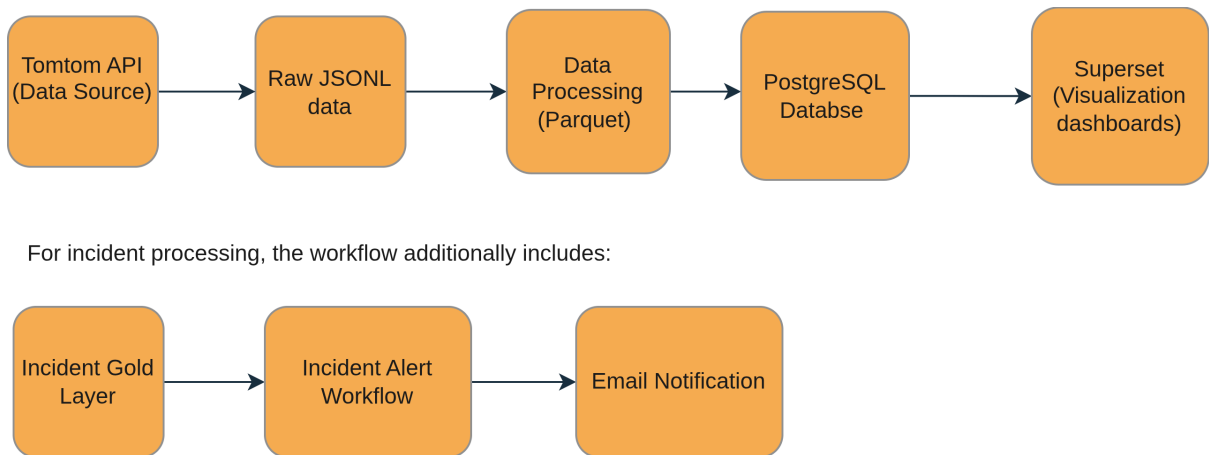


Figure 3.4: pipeline execution flow

Chapter 4

System Implementation

4.1 Implementation Overview

This chapter presents the implementation of the ETL platform based on the architecture introduced in Chapter 3. As mentioned in the design, the process combines Apache Airflow for workflow orchestration, Apache Spark for distributed data transformation, PostgreSQL for analytical storage, Apache Superset for dashboard visualization, Docker Compose for containerized execution, and Terraform for infrastructure deployment.

The pipeline follows a modular structure that separates data acquisition, orchestration, transformation, storage, monitoring, visualization, and deployment. This structure reflects the actual organization of the project and supports maintainability, reproducibility, and independent recovery of workflow stages.

4.2 Data Acquisition and API Integration

The data collection layer collects real-time traffic flow and incident data from the TomTom Traffic API. Python-based extraction functions are used to communicate with the external API services and store the retrieved responses as raw JavaScript Object Notation Lines (JSONL) batch files for Spark processing.

Two TomTom services are integrated: the Flow Segment API, which is used to collect traffic flow data within each configured city, and the Incident Details API, which is used to retrieve traffic incident records. The raw API responses contain the source variables that are preserved during the data acquisition stage and subsequently used as inputs to the transformation process. Tables 4.1 and 4.2 summarize the main variables contained in the raw traffic flow and incident data before any transformation or standardization is applied. respectively.

Table 4.1: Raw traffic flow variables collected during data acquisition

Variable	Data Type	Nullability	Possible/Example Values
city	String	Not Null	Berlin, Bremen, Frankfurt
batch_ts	String	Not Null	e.g., 20260721120000
lat	Double	Not Null	e.g., 52.5200°
lon	Double	Not Null	e.g., 13.4050°
frc	Integer	Nullable	Functional Road Class value
current_speed	Integer	Nullable	Traffic speed in km/h
free_flow_speed	Integer	Nullable	Free-flow speed in km/h
travel_time	Integer	Nullable	Current travel time in seconds
confidence	Double	Not Null	Numeric confidence value
road_closure	Boolean	Not Null	true, false

Table 4.2: Raw incident variables collected during data acquisition

Variable	Data Type	Nullability	Possible/Example Values
city	String	Not Null	Berlin, Bremen, Frankfurt
batch_ts	String	Not Null	e.g., 20260721120000
incident_type	String	Nullable	Traffic incident feature
category	Integer	Nullable	1: Accident; 2: Fog; 3: Dangerous Conditions; 4: Rain; 5: Ice; 6: Jam; 7: Lane Closed; 8: Road Closed; 9: Road Works; 10: Wind; 11: Flooding; 14: Broken Down Vehicle
lat	Double	Not Null	e.g., 52.5200
lon	Double	Not Null	e.g., 13.4050
delay_seconds	Integer	Nullable	e.g., 60, 180, 600
road_numbers	String	Not Null	e.g., ["A1"], ["B6"], ["A27"]
from_road	String	Nullable	Road or location name
to_road	String	Nullable	Road or location name
start_time	String	Nullable	ISO 8601 timestamp
end_time	String	Nullable	ISO 8601 timestamp

The records contain variables extracted from the TomTom API together with common pipeline metadata used to support multi-city processing and batch identification. The `city` field identifies the configured city associated with each record, while `batch_ts` identifies the acquisition batch. The remaining fields represent the traffic flow or incident information extracted from the corresponding TomTom API response. Figure 4.1 shows an example of a traffic flow record stored in JSONL format during the data acquisition stage. The record contains the traffic flow variables described in Table 4.1.

File Name	JSON Object
tomtom_berlin_20260721T113000.jsonl	{"city": "bremen", "batch_ts": "20260720T191500", "lat": 53.03, "lon": 8.72, "frc": "FRC"}
tomtom_berlin_20260721T114500.jsonl	{"city": "bremen", "batch_ts": "20260720T191500", "lat": 53.063333, "lon": 8.72, "frc": "FRC"}
tomtom_berlin_20260804T201500.jsonl	{"city": "bremen", "batch_ts": "20260720T191500", "lat": 53.096667, "lon": 8.72, "frc": "FRC"}
tomtom_berlin_20260804T203000.jsonl	{"city": "bremen", "batch_ts": "20260720T191500", "lat": 53.13, "lon": 8.72, "frc": "FRC"}
tomtom_berlin_20260804T204500.jsonl	{"city": "bremen", "batch_ts": "20260720T191500", "lat": 53.03, "lon": 8.796667, "frc": "FRC"}
tomtom_bremen_20260720T191500.jsonl	{"city": "bremen", "batch_ts": "20260720T191500", "lat": 53.063333, "lon": 8.796667, "frc": "FRC"}
tomtom_bremen_20260720T193000.jsonl	{"city": "bremen", "batch_ts": "20260720T191500", "lat": 53.096667, "lon": 8.796667, "frc": "FRC"}
tomtom_bremen_20260720T193951.jsonl	{"city": "bremen", "batch_ts": "20260720T191500", "lat": 53.13, "lon": 8.796667, "frc": "FRC"}
tomtom_bremen_20260720T194500.jsonl	{"city": "bremen", "batch_ts": "20260720T191500", "lat": 53.03, "lon": 8.873333, "frc": "FRC"}
tomtom_bremen_20260721T111500.jsonl	{"city": "bremen", "batch_ts": "20260720T191500", "lat": 53.063333, "lon": 8.873333, "frc": "FRC"}
tomtom_bremen_20260721T113000.jsonl	{"city": "bremen", "batch_ts": "20260720T191500", "lat": 53.096667, "lon": 8.873333, "frc": "FRC"}
tomtom_bremen_20260721T113000.jsonl	{"city": "bremen", "batch_ts": "20260720T191500", "lat": 53.13, "lon": 8.873333, "frc": "FRC"}
tomtom_bremen_20260721T113000.jsonl	{"city": "bremen", "batch_ts": "20260720T191500", "lat": 53.03, "lon": 8.95, "frc": "FRC"}
tomtom_bremen_20260721T113000.jsonl	{"city": "bremen", "batch_ts": "20260720T191500", "lat": 53.063333, "lon": 8.95, "frc": "FRC"}
tomtom_bremen_20260721T113000.jsonl	{"city": "bremen", "batch_ts": "20260720T191500", "lat": 53.096667, "lon": 8.95, "frc": "FRC"}

Figure 4.1: Sample raw traffic data collected in JSONL format

4.3 Workflow Orchestration

The orchestration layer is implemented using five Apache Airflow DAGs written in Python. The DAGs separate traffic ingestion, traffic processing, Gold aggregation, incident processing, and incident alerting into independently manageable workflows. Airflow is responsible for scheduling, dependency management, triggering Spark jobs, and monitoring their execution, while the actual data transformation is performed by Apache Spark.

The traffic flow workflow starts with collecting traffic data every 15 minutes. After successful ingestion, Airflow triggers the Spark job responsible for the Bronze and Silver transformations, and finally, the Gold workflow is triggered after every four Spark processing runs (Bronze and Silver jobs), corresponding to an hourly aggregation cycle. Airflow does not perform these transformations itself; it only orchestrates.

The incident workflow operates independently; it collects incident data every 15 minutes and triggers the corresponding Spark processing stages. After successful incident Gold processing, an alert is triggered to evaluate critical incidents and generate email notifications.

The overall pipeline orchestrations are illustrated in Figures 4.3 and 4.4, respectively. An overview of the five implemented DAGs is shown in Figure 4.2.

DAG	Owner	Runs	Schedule	Last Run	Next Run	Recent Tasks	Actions	Links
incident_alert_dag	Bihane	1	None	2026-08-12, 03:07:53				
tomtom_api_ingestion	Bihane	1	*15 ***	2026-08-12, 02:45:00	2026-08-12, 03:00:00			
tomtom_gold_dag	Bihane	1	None	2026-08-04, 20:43:42				
tomtom_incidents_pipeline	Bihane	1	*15 ***	2026-08-12, 02:45:00	2026-08-12, 03:00:00			
tomtom_spark_pipeline	Bihane	1	None	2026-08-12, 03:06:09				

Figure 4.2: Overview of the five implemented Airflow DAGs

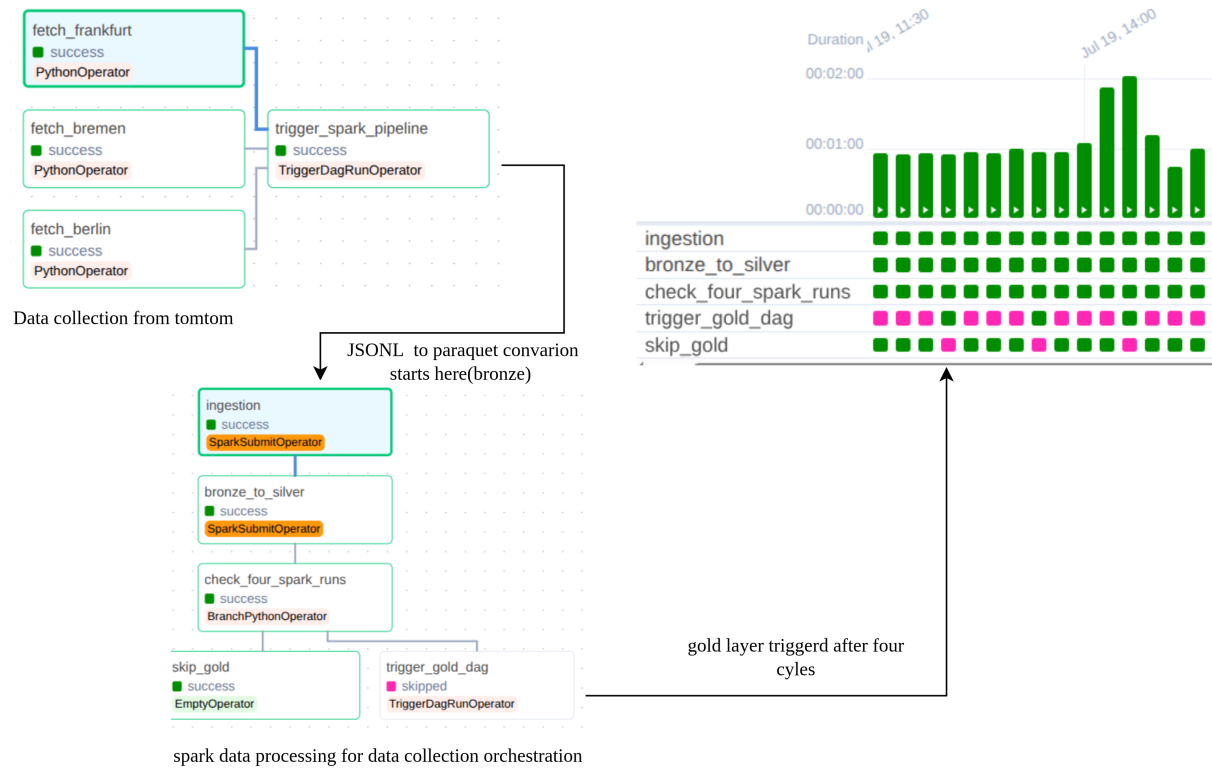


Figure 4.3: Airflow orchestration for traffic flow pipeline

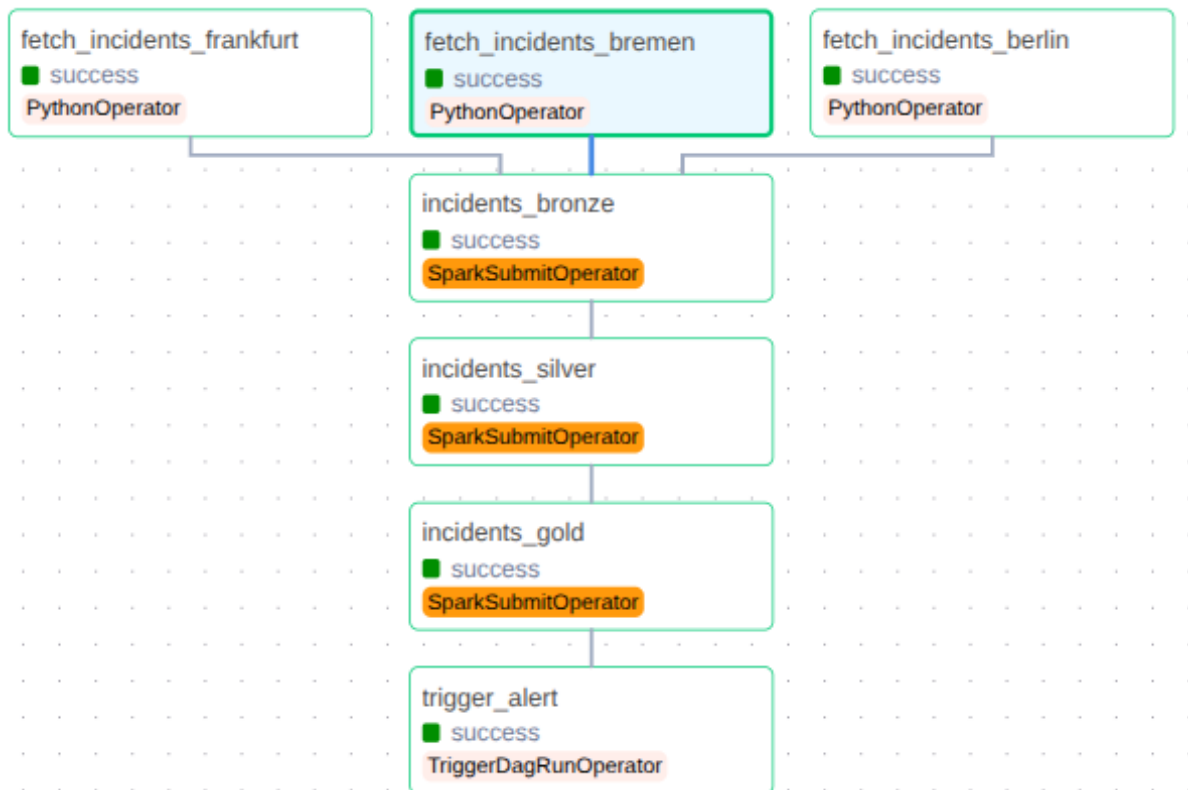


Figure 4.4: Airflow orchestration for incidents pipeline

Table 4.3 summarizes the responsibilities, schedules, and trigger relationships of the

five implemented DAGs.

Table 4.3: Airflow DAG responsibilities and trigger relationships

DAG	Responsibility	Trigger	Next DAG / Job
tomtom_api_ingestion	Collects traffic flow data for all configured cities and stores the raw ingestion batches.	Every 15 min	Spark processing
tomtom_spark_pipeline	Submits and monitors Spark jobs for Bronze and Silver traffic transformations.	After ingestion	Gold after 4 runs
tomtom_gold_dag	Submits Spark jobs for Gold traffic transformation and hourly KPI aggregation.	After 4 runs	None
tomtom_incidents_pipeline	Collects incident data and submits the Spark jobs for Bronze, Silver, and Gold incident processing.	Every 15 min	incident_alert_dag
incident_alert_dag	Evaluates processed incidents and generates email notifications for critical events.	After incident Gold	None

Self-healing, one of research objectives is implemented within the Airflow orchestration layer through automatic retries, exponential backoff, failure isolation, and decoupled DAG execution. Airflow tasks are configured with three retries and exponential backoff, allowing transient failures to be retried automatically without manual intervention. Failure isolation is also applied. A failed API request therefore does not terminate the complete city ingestion, while the failure is recorded as a dropped record.

The DAGs are connected to keep downstream processing independent from the scheduling of upstream ingestion. Consequently, a failure in a downstream Spark workflow does not prevent the next 15-minute ingestion cycle from running. This decoupling limits failure propagation and reduces the risk of cascading failures across the pipeline.

4.4 Distributed Data Processing

4.4.1 Medallion data processing architecture

Apache Spark is used as the distributed processing engine for transforming raw traffic and incident data into analytical datasets. In the Bronze layer, Spark reads the raw JSONL files generated by the ingestion workflows and converts them into structured Parquet datasets. For traffic flow data, the Bronze job applies schema parsing, filters invalid

geographic records, adds ingestion metadata, and stores the result in partitioned Parquet format. A similar Bronze job is implemented for incident data.

In the Silver layer, Spark performs data refinement and feature transformation. For traffic flow data, this includes null filtering, computation of the `speed_ratio`, timestamp handling, and deduplication using window-based logic. For incident data, the Silver layer standardizes timestamps, preserves geographic information, and prepares the records for analytical use.

In the Gold layer, Spark generates dashboard-ready datasets. The traffic Gold job performs hourly aggregation over the most recent hour of Silver data and writes results to both Parquet and PostgreSQL. It produces point-level traffic outputs for spatial visualization and Key Performance Indicator (KPI)-level outputs for analytical reporting. The incident Gold job selects the latest processed incident records and loads them into PostgreSQL for dashboard evaluation.

4.4.2 Data Cleaning and Validation

Before transformation, the ingested datasets undergo a data cleaning stage. Duplicate records are removed, missing or invalid values are handled according to predefined rules, and timestamps are converted into a consistent format. Geographic coordinates and numerical traffic measurements are validated to ensure that only meaningful observations continue to subsequent processing stages. These validation steps improve data quality and reduce the propagation of inconsistent records into analytical datasets.

4.4.3 Feature Engineering

The raw data collected represent the initial schema received from the external data source before any transformation or feature engineering operations are applied. During Spark-based processing, the collected variables are cleaned, standardized, and enriched with additional analytical attributes. Tables 4.4 and 4.5 summarize the feature-engineered variables for traffic flow and incidents, respectively.

Table 4.4: Traffic flow feature engineering variables

Feature	Data Type	Calculation and Purpose
Speed Ratio	Double	Calculated as the ratio between current speed and free-flow speed: $SpeedRatio = \frac{CurrentSpeed}{FreeFlowSpeed}$
Speed Difference	Double	Represents the percentage of normal speed maintained by a road segment. Calculated as: $SpeedDifference = FreeFlowSpeed - CurrentSpeed$
Travel Time Ratio	Double	Measures the reduction in speed caused by traffic congestion. Calculated as: $TravelTimeRatio = \frac{CurrentTravelTime}{FreeFlowTravelTime}$
Congestion Level	String	Indicates additional travel time compared with normal traffic conditions. Derived from SpeedRatio thresholds to classify traffic conditions (e.g., low, moderate, and high congestion).

Table 4.5: Traffic incident feature engineering variables

Feature	Data Type	Calculation and Purpose
Incident Duration	Integer	Calculated as: $IncidentDuration = ProcessingTime - IncidentStartTime$
Severity Score	Integer	Measures the active duration of a reported incident. Converts categorical severity levels into numerical values: $SeverityScore = \begin{cases} 1, & Low \\ 2, & Medium \\ 3, & High \end{cases}$
Processing Date	Date	Supports quantitative incident analysis. Extracted from the processing timestamp and used for partitioning and temporal analysis.
Batch ID	String	Generated during ETL execution to maintain data lineage across micro-batches.

4.4.4 Data Partitioning Strategy

To improve processing efficiency, the Parquet datasets are partitioned according to city and processing date. This organization enables Apache Spark to read only the required partitions during analytical queries instead of scanning the complete dataset. Partition pruning reduces disk I/O, shortens execution time, and improves scalability as the data volume increases.

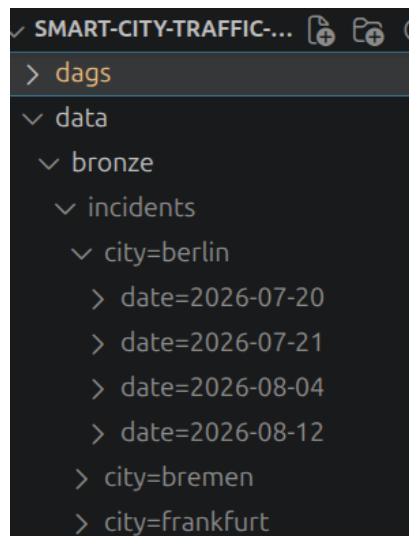


Figure 4.5: Data partition based on date and city

4.5 Multi-City Data Processing Implementation

One objective of the ETL pipeline is to process traffic data from multiple cities using a single implementation. Instead of developing separate workflows for each location, the pipeline uses a configuration-driven approach in which city-specific parameters are separated from the common processing logic. This allows the same Airflow workflows and Spark transformation jobs to be reused across Berlin, Bremen, and Frankfurt.

4.5.1 City Configuration and Grid Generation

The multi-city implementation is managed through centralized configuration structure. As shown in Figure 4.6, each city is defined by a geographic bounding box (**bbox**) and a **grid_steps** parameter. The bounding box defines the geographic area used for traffic data collection, while **grid_steps** determines the number of sampling divisions along each dimension.


```

CITY_CONFIG = {
    "berlin": {"bbox": (13.28, 52.46, 13.57, 52.57), "grid_steps": 5},
    "bremen": {"bbox": (8.72, 53.03, 8.95, 53.13), "grid_steps": 4},
    "frankfurt": {"bbox": (8.55, 50.05, 8.80, 50.18), "grid_steps": 5},
}

```

Figure 4.6: Configuration structure used for multi-city traffic data collection

The implementation generates a square grid of sampling points within each configured bounding box. Therefore, the total number of sampling points is calculated as:

$$N = \text{grid_steps} \times \text{grid_steps} \quad (4.1)$$

Based on the configurations shown in Figure 4.6, Berlin uses five grid steps and therefore generates

$$N_{Berlin} = 5 \times 5 = 25 \quad (4.2)$$

sampling points. Bremen uses four grid steps and generates

$$N_{Bremen} = 4 \times 4 = 16 \quad (4.3)$$

sampling points, while Frankfurt uses five grid steps and generates

$$N_{Frankfurt} = 5 \times 5 = 25 \quad (4.4)$$

sampling points.

The generated sampling points are used to construct the traffic flow API requests for each city during every ingestion cycle. The same city configuration is subsequently used by the downstream processing stages, allowing the ETL workflow and Spark transformation logic to remain unchanged when additional cities are added.

4.6 Data Warehouse, Storage, and Monitoring

The pipeline uses Parquet for the Bronze, Silver, and Gold processing layers and PostgreSQL for structured analytical access. This separation allows Spark to process intermediate datasets independently from the analytical database.

Figure 4.7 shows the implemented data flow from data acquisition and processing to analytical storage and monitoring.

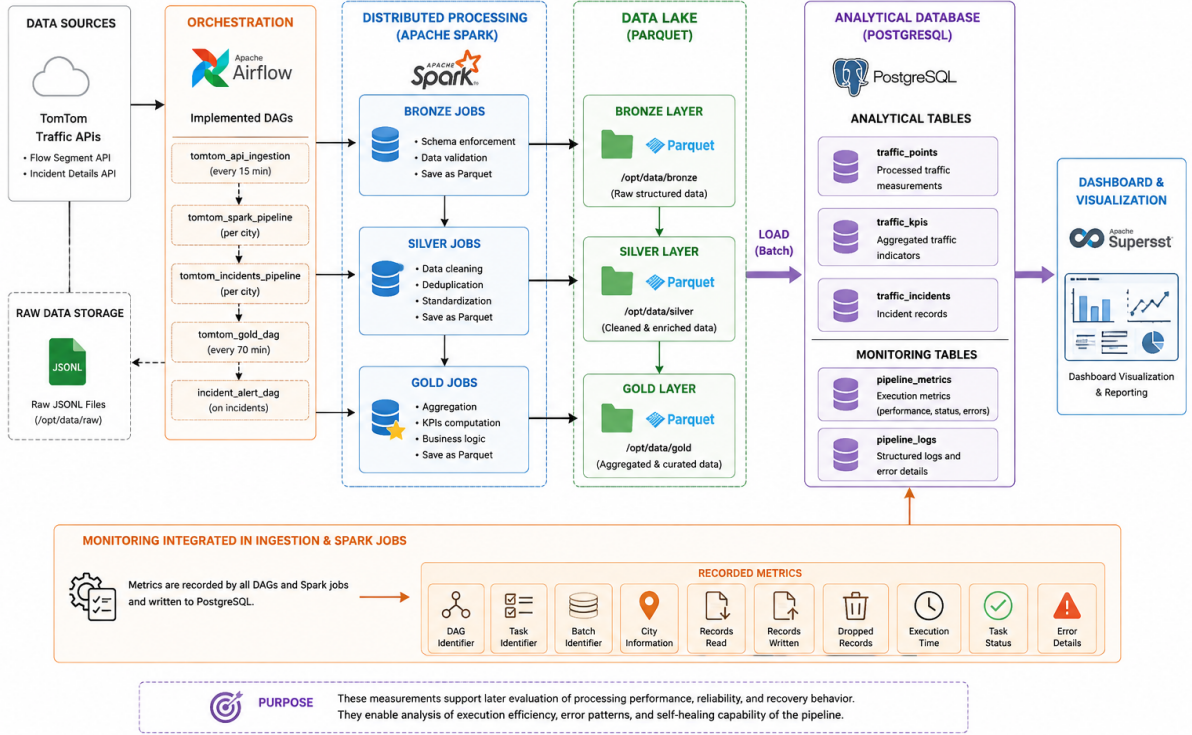


Figure 4.7: Overall architecture of the implemented pipeline with data storage, analytical warehouse, and monitoring components.

The PostgreSQL database contains the main analytical tables `traffic_points`, `traffic_kpis`, and `traffic_incidents`, which store processed traffic measurements, aggregated traffic indicators, and incident records, respectively.

Pipeline execution is monitored through the `pipeline_metrics` and `pipeline_logs` tables. The recorded information includes batch and task identifiers, city, records processed and dropped, execution time, task status, errors, file paths, and retry information. These records are used in Chapter 5 to evaluate pipeline performance, failures, and recovery behaviour.

4.7 Data Visualization Dashboards

Apache Superset provides analytical access to the PostgreSQL datasets. Three dashboard views are implemented for traffic flow, traffic incidents, and pipeline monitoring.

The traffic flow dashboard uses `traffic_points` and `traffic_kpis` to present traffic conditions, including speed, speed ratio, congestion level, and city-based comparisons. The incident dashboard uses `traffic_incidents` to present incident types, affected roads, locations, and delay information. The pipeline monitoring dashboard uses `pipeline_metrics` and `pipeline_logs` to present execution time, processing volume, failures, and retries.

These dashboards provide the analytical interface for inspecting the outputs and operational behaviour of the implemented pipeline.

Sample dashboards are shown in Figures 4.8, 4.9, and 4.10.

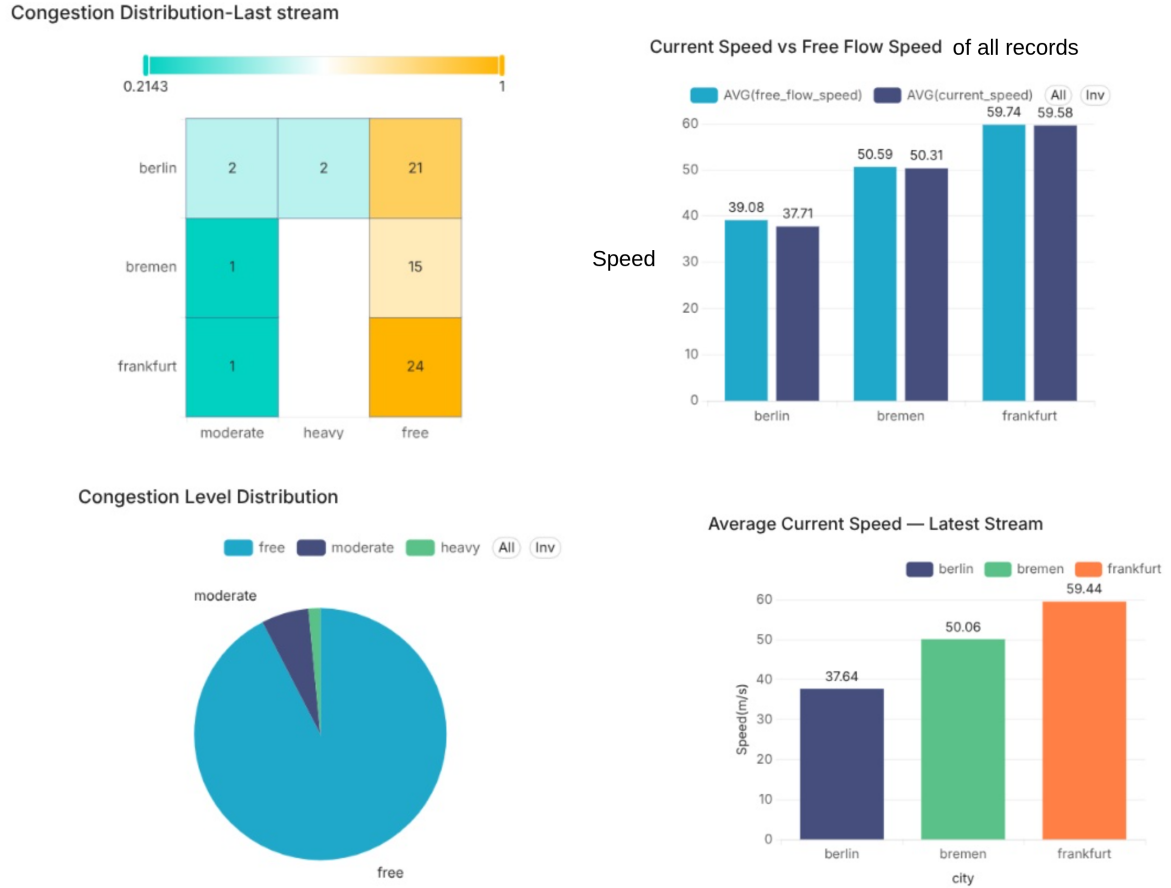


Figure 4.8: Traffic flow visualization dashboards

The visualizations in the traffic flow dashboard answer the following sample analytical questions:

1. *Congestion Distribution – Last Stream* answers: How are traffic congestion levels distributed across the monitored cities in the latest data stream?
2. *Current Speed vs Free-Flow Speed of All Records* answers: How does the current traffic speed compare with the corresponding free-flow speed across the evaluated cities?
3. *Congestion Level Distribution* answers: What proportion of traffic observations falls into each congestion level?
4. *Average Current Speed – Latest Stream* answers: Which city has the highest and lowest average current traffic speed in the latest stream?

5. *Current Speed vs Free Flow Speed of All Records* also answers: Which city appears to experience greater traffic congestion based on the difference between current and free-flow speeds?
6. The *Congestion Distribution – Last Stream* and *Current Speed vs Free Flow Speed* visualizations collectively answer: How does the congestion situation differ between Berlin, Bremen, and Frankfurt?

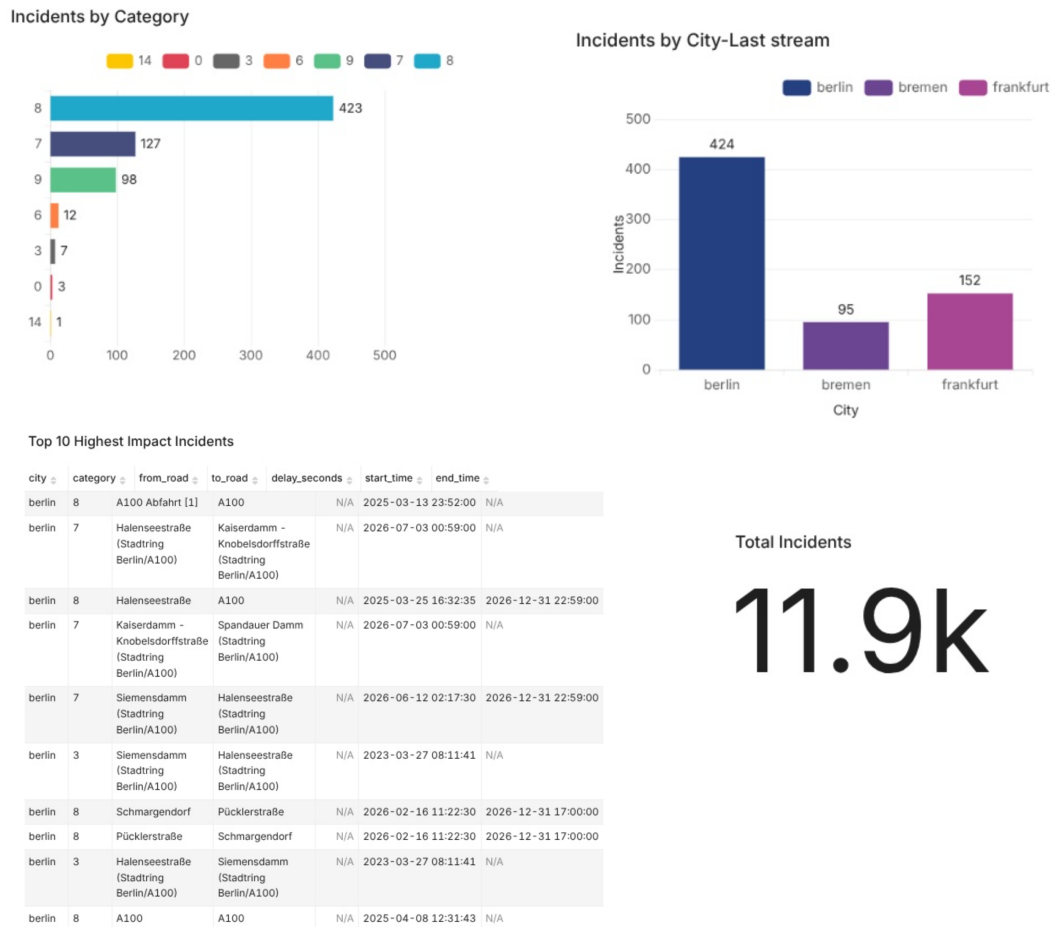


Figure 4.9: Traffic incidents visualization dashboards



Figure 4.10: Pipeline evaluation and monitoring dashboards

The dashboards therefore provide both an analytical view of the processed traffic data and a monitoring view of the ETL pipeline itself. This connects the final traffic outputs with the execution information collected during pipeline processing. one dashboard answers many analytical questions, like given below figure 4.8.

4.8 Alerting Implementation

The alerting component's notification mechanism was implemented using email delivery over SMTP rather than push notifications, SMS, or a dedicated messaging platform. This choice reflects a common pattern in data engineering monitoring practice, where email remains a low-overhead, infrastructure-agnostic channel for delivering event-triggered notifications without requiring additional third-party service integration [7]. Because the alert workflow is triggered only after the incident Gold layer confirms new critical-category records, this design follows an event-driven rather than polling-based notification pattern, ensuring that notifications are generated only when the underlying data justifies one, reducing unnecessary alert volume.

A practical constraint encountered during implementation was that major email providers, including Gmail, have progressively restricted authentication for third-party SMTP clients, replacing static account passwords with application-scoped credentials generated under two-factor authentication. This reflects a broader industry shift toward reducing creden-

tial exposure in automated systems, where long-lived plaintext passwords are increasingly disfavoured in favour of scoped, revocable tokens. The alert workflow's SMTP authentication was therefore configured using such an application-scoped credential rather than the account's primary password, consistent with this security practice. The credential itself was stored as an Airflow connection object rather than embedded in DAG source code, separating sensitive configuration from version-controlled application logic and supporting the reproducibility objective discussed in Section 5.2, since the connection can be reconfigured per deployment environment without modifying the pipeline's codebase.

Figure 4.11 shows a delivered alert email generated by the workflow, confirming that the notification mechanism correctly surfaces critical incident information, including affected location, category, and detection time, to the end user without manual intervention.



Figure 4.11: Incident alert email notification

4.9 Infrastructure and Container Deployment

The environment was first deployed locally using Docker Compose to execute and evaluate the complete pipeline. As shown in Figure 4.12, docker compose up -d starts all required containers: PostgreSQL for data storage, Apache Spark (master and worker) for data processing, Apache Airflow for orchestration (initialization, scheduler, and webserver) for workflow orchestration, and Apache Superset for data visualization.

Docker Containerized images

```

birhane@Bbirhane-Notebook:~/Videos/smart-city-traffic-etl$ docker compose up -d
[+] up 7/7
✓ Container smart-city-traffic-etl-postgres-1      Started      0.2s
✓ Container spark-master                          Started      0.2s
✓ Container spark-worker                          Started      0.1s
✓ Container smart-city-traffic-etl-airflow-init-1  Started      0.1s
✓ Container smart-city-traffic-etl-superset-1     Started      0.2s
✓ Container smart-city-traffic-etl-airflow-webserver-1 Started      0.0s
✓ Container smart-city-traffic-etl-airflow-scheduler-1 Started      0.0s
  
```

Figure 4.12: Startup of the Docker Compose environment

The successful startup of all containers confirms that the ETL environment is correctly configured and ready to execute the pipeline.

This approach simplifies development and testing by keeping all platform components in a single deployment definition. However, Docker Compose alone does not provision underlying cloud infrastructure. To support cloud infrastructure-level reproducibility, Terraform is used to provision AWS EC2 resources required for deployment.

In general, the resulting deployment process consists of two complementary layers: Terraform provisions the infrastructure, while Docker Compose provisions the application services within that infrastructure.

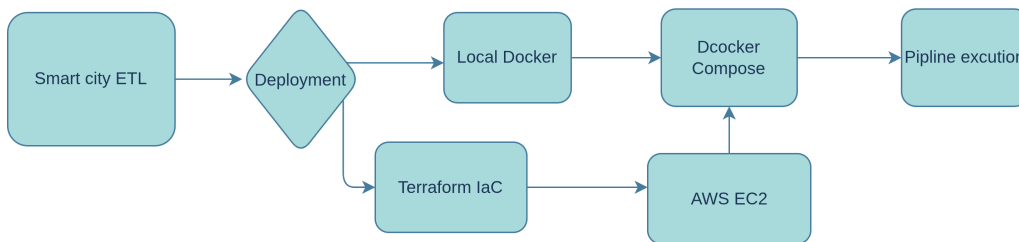


Figure 4.13: Local vs Terraform-based deployment of the Dockerized ETL environment

As shown in Figure 4.13, Terraform provisions the AWS EC2 instance, security group, Secure Shell (SSH) key pair used to connect to the local machine, and storage. During instance initialization, the Terraform bootstrap script installs Docker and Docker Compose, clones the project from GitHub[24], and starts the Docker Compose environment;

it does not require the application services to be installed individually on the host (EC2) operating system.

The AWS deployment used Ubuntu 24.04 in the Frankfurt region (`eu-central-1`) on a `t3.small` EC2 instance with a 20 GB GP3 root volume. The instance type can be scaled to larger instances when required.

Manual Deployment versus Infrastructure as Code

Two approaches were compared. The manual approach requires the operator to create and configure the Amazon Machine Image (AMI), EC2 virtual machine, configure the security group, configure SSH access, install Docker and Docker Compose, retrieve the project, configure the environment, and start the Docker Compose services. The Terraform approach automates the infrastructure provisioning and initial environment setup through the version-controlled Terraform configuration and bootstrap process.

Table 4.6: Infrastructure provisioning time comparison

Infrastructure activity	Manual deployment	Terraform deployment
configure Ubuntu AMI	10 min	1 min
create EC2 instance type	5 min	0.5 min
Configure storage	8 min	0.5 min
Configure network/VPC settings	8 min	1 min
Configure security group	10 min	1 min
Configure SSH key pair	5 min	0.5 min
Launch/provision EC2 instance	5 min	3 min
Verify SSH access	5 min	2 min
Clone repository	5 min	1 min
Total	61 min	10.5 min

The manual deployment requires multiple configuration steps and depends on operator intervention, whereas Terraform represents infrastructure configuration as reusable code and automates provisioning.

This demonstrates that the Terraform-based approach reduces the estimated effort required to reproduce the deployment environment from approximately 61 minutes to 10.5 minutes:

$$\text{Deployment Effort Reduction (\%)} = \frac{T_{\text{manual}} - T_{\text{Terraform}}}{T_{\text{manual}}} \times 100$$

$$= \frac{61 - 10.5}{61} \times 100 \approx 82.8\%$$

4.9.1 Source Code Repository

The complete implementation of the smart-city traffic ETL pipeline is maintained in a version-controlled GitHub repository [24]. The repository provides the implementation and deployment artifacts, including the Airflow DAGs, Spark processing jobs, Docker Compose configuration, Terraform infrastructure definitions, configuration files, tests, and deployment documentation. This artifact also enables the implemented environment and pipeline configuration to be recreated independently of the original development environment.

4.9.2 Implementation Challenges and Resolutions

During implementation, several practical issues were encountered and resolved. These issues informed design decisions related to API integration, data processing, orchestration, and deployment.

TomTom API request encoding. The Incident Details API required specific URL characters to remain unencoded. This was resolved by using `urllib.request` to construct requests with explicit URL handling.

PySpark naming conflict. A conflict between Python's `round()` and PySpark's `round()` functions caused incorrect function resolution. This was resolved using qualified PySpark imports.

Airflow email configuration. The deployed Airflow environment did not correctly use the configured SMTP environment variables. An explicit `smtp_default` connection was therefore configured through the Airflow interface.

Superset database configuration. Superset initially failed to connect to PostgreSQL because of an incorrect database configuration variable. Updating the variable to the format expected by the deployed Superset image resolved the issue.

Container and provider configuration. The Spark workflow required explicit installation of the Spark provider, while an incorrect Docker Compose health-check command caused repeated container startup failures. Correcting both configurations resolved the deployment issues.

These challenges and their resolutions contributed to the final design of the pipeline, particularly its monitoring, failure handling, and deployment configuration.

Chapter 5

Evaluation and Discussion

This chapter evaluates the implemented Smart City Traffic ETL pipeline against the research questions defined in Chapter 1. The evaluation focuses on reproducibility, resilience, and reusability across multiple cities. In addition, the deployment approach is evaluated by comparing manual AWS ETL deployment infrastructure provisioning with the proposed Terraform-based approach.

5.1 Evaluation Strategy

The evaluation considers the complete data flow from TomTom API ingestion through the Bronze, Silver, and Gold processing layers to PostgreSQL, Superset visualization, and alerting components.

The following criteria were used:

- **Reproducibility:** ability to recreate the application and infrastructure environment using Docker Compose and Terraform.
- **Resilience:** ability of the Airflow-based pipeline to recover from or handle task failures and continue processing.
- **Reusability:** ability to process data from multiple cities using a common configuration and standardized schemas.
- **Deployment effort:** time and manual effort required to provision the deployment environment.
- **Data processing performance:** number of records processed, execution time, and processing outcomes across pipeline stages.

5.1.1 Functional and Unit Testing

The configuration and validation functions were tested using `pytest`. Three unit tests were implemented for `CITY_CONFIG` and `validate_city_config()`. The tests check whether the expected cities and configuration values are present, whether a valid configuration is accepted, and whether an invalid configuration is rejected.

Table 5.1: Functional and unit test results

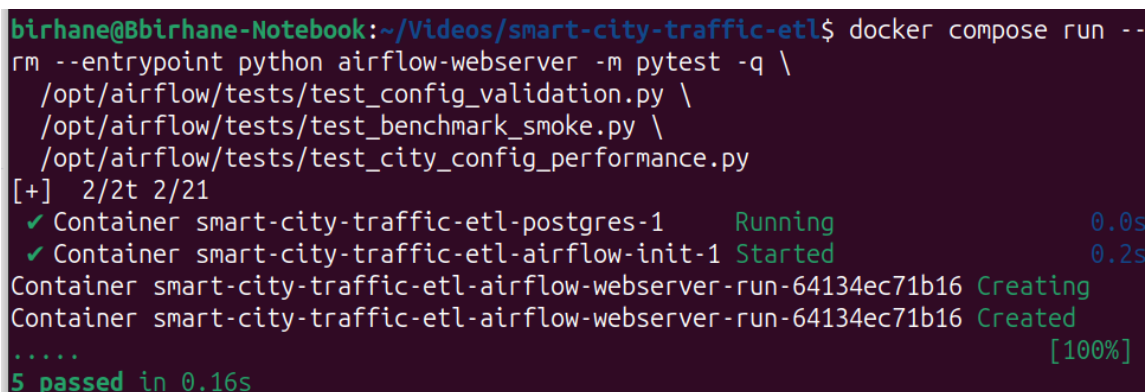
Test	Category	Result
<code>test_city_config_contains_expected_cities</code>	Unit	PASS
<code>test_validate_city_config_accepts_valid_input</code>	Unit	PASS
<code>test_validate_city_config_rejects_invalid_input</code>	Unit	PASS
Total	3 tests	3 PASS

All three configuration tests passed successfully. The invalid configuration test also confirmed that malformed input raises the expected `ValueError`.

5.1.2 Performance Benchmarking

Two performance-related tests were also executed. The benchmark smoke test checks that a simple operation completes within the defined time limit. The second test measures the execution time of `build_grid()` for the three configured cities.

Together with the three unit tests, all five tests passed successfully. The complete test run took 0.16 seconds, as shown in Figure 5.1.



```

birhane@Bbirhane-Notebook:~/Videos/smart-city-traffic-etl$ docker compose run --rm --entrypoint python airflow-webserver -m pytest -q \
/opt/airflow/tests/test_config_validation.py \
/opt/airflow/tests/test_benchmark_smoke.py \
/opt/airflow/tests/test_city_config_performance.py
[+] 2/2t 2/21
✓ Container smart-city-traffic-etl-postgres-1 Running 0.0s
✓ Container smart-city-traffic-etl-airflow-init-1 Started 0.2s
Container smart-city-traffic-etl-airflow-webserver-run-64134ec71b16 Creating
Container smart-city-traffic-etl-airflow-webserver-run-64134ec71b16 Created
..... [100%]
5 passed in 0.16s

```

Figure 5.1: Execution of the automated test suite

Table 5.2 shows the measured grid-generation times. Berlin generated 25 points in 0.000022 seconds, Bremen generated 16 points in 0.000012 seconds, and Frankfurt gen-

erated 25 points in 0.000015 seconds. In total, 66 points were generated in 0.000080 seconds.

Table 5.2: City-grid generation performance

City	Grid Points	Execution Time (s)
Berlin	25	0.000022
Bremen	16	0.000012
Frankfurt	25	0.000015
Total	66	0.000080

The benchmark result shows that grid generation takes very little time for the three configured cities. Figure 5.2 shows the corresponding terminal output.

```

birhane@Bbirhane-Notebook:~/Videos/smart-city-traffic-etl$ docker compose run --
rm --entrypoint python airflow-webserver -m pytest -s -q \
/opt/airflow/tests/test_city_config_performance.py
[+] 2/2t 2/21
✓ Container smart-city-traffic-etl-postgres-1 Running 0.0s
✓ Container smart-city-traffic-etl-airflow-init-1 Started 0.1s
Container smart-city-traffic-etl-airflow-webserver-run-930277cd67f1 Creating
Container smart-city-traffic-etl-airflow-webserver-run-930277cd67f1 Created
berlin: 0.000022 seconds, 25 points
bremen: 0.000012 seconds, 16 points
frankfurt: 0.000015 seconds, 25 points
Total: 0.000080 seconds, 66 points

```

Figure 5.2: City-grid generation benchmark for Berlin, Bremen, and Frankfurt

This benchmark covers the grid-generation component only. The performance of the complete ETL pipeline is evaluated separately using the Airflow and Spark execution metrics.

5.1.3 Integration Testing

Integration testing was used to verify the interaction between the reusable city configuration and the grid-generation component. Unlike the unit tests, which test the configuration validation functions individually, the integration test uses the city configurations from `CITY_CONFIG` as input to `build_grid()` and verifies that valid grid points are generated for each configured city.

The test covers the interaction between the configuration data and the grid-generation function for Berlin, Bremen, and Frankfurt. For each city, the configured bounding box and grid-step value are passed to `build_grid()`. The generated points are then checked to ensure that they are within the corresponding city bounding box.

Table 5.3: Integration test result

Integration test	Components	Result
City configuration to grid generation	CITY_CONFIG + build_grid()	PASS

The integration test passed successfully, confirming that the configurations defined in CITY_CONFIG can be used by build_grid() to generate valid grid points for all three configured cities.

```

birhane@Bbirhane-Notebook:~/Videos/smart-city-traffic-etl$ docker compose run --entrypoint python airflow-webserver -m pytest -q \
/opt/airflow/tests/test_config_validation.py \
/opt/airflow/tests/test_benchmark_smoke.py \
/opt/airflow/tests/test_city_config_performance.py \
/opt/airflow/tests/test_city_config_integration.py
[+] 2/2t 2/21
✓ Container smart-city-traffic-etl-postgres-1 Running 0.0s
✓ Container smart-city-traffic-etl-airflow-init-1 Started 0.1s
Container smart-city-traffic-etl-airflow-webserver-run-b530fd0606ac Creating
Container smart-city-traffic-etl-airflow-webserver-run-b530fd0606ac Created
..... [100%]
6 passed in 0.15s

```

Figure 5.3: Successful execution of the city configuration integration test

Figure 5.3 shows the successful execution of the integration test, which completed with one passing test.

5.2 Reproducibility Evaluation

Reproducibility was evaluated at two levels. At the application level, Docker Compose defines the required ETL services, configurations, and dependencies in a single YAML file. This allows the application environment to be recreated on another machine without manually installing and configuring individual components.

At the infrastructure level, Terraform defines the AWS resources required to host the ETL platform, providing a repeatable deployment process and reducing manual configuration. The comparison between manual and Terraform-based deployment is presented in Table 4.6.

The Terraform-based approach reduced the estimated deployment effort by approximately 82.8%. Terraform provides reproducible infrastructure, while Docker Compose provides a consistent application environment and service dependencies. Together, they reduce environment-specific configuration and support recreation of the ETL pipeline across different execution environments.

5.3 Pipeline Self Healing Evaluation

Self-healing refers to the pipeline’s ability to automatically detect task failures and apply predefined recovery actions, primarily through Airflow task retries and exponential backoff. Resilience was evaluated using a controlled fault-injection test and the results of 270 DAG runs. The results are summarized in Table 5.4.

Table 5.4: Airflow DAG resilience evaluation

(a) Controlled failure and automatic recovery		(b) Observed DAG execution results	
Evaluation Metric	Result	Metric	Result
Failure introduced	Yes	Total DAG runs	270
Initial task attempt	Failed	Successful DAG runs	260
Automatic retry triggered	Yes	Failed DAG runs	10
Manual intervention required	No	Recovered after retry	3
Retry attempt	Successful	Unrecoverable plugin-related failures	7
Downstream tasks completed	Yes		
Final DAG status	Successful		

The fault-injection test confirmed that a transient task failure was automatically recovered through the configured retry mechanism without manual intervention. During regular operation, 260 of 270 DAG runs completed successfully. Three failed runs were recovered after retry, while seven plugin-related failures remained unrecoverable because they were outside the retry mechanism’s recovery scope.

5.4 Multi-City Reusability Evaluation

Reusability was evaluated by executing the same ETL architecture for Berlin, Bremen, and Frankfurt. Instead of implementing separate processing logic for each city, the pipeline uses shared processing components and city-specific configuration.

The evaluation therefore examined whether the same ingestion and Spark processing logic could process data from all three cities while maintaining a common schema.

Table 5.5: Multi-city processing evaluation

Metric	Berlin	Bremen	Frankfurt
Records ingested	6.86k	6.86k	6.86k
Records processed	2.33k	1.49k	2.33k
Records dropped	728	726	726

The results demonstrate that the same ETL processing logic was applied to all three cities while accommodating city-specific data characteristics through configuration. The

differences in the numbers of processed and dropped records reflect variations in the data available for each city and the filtering applied during processing. This provides quantitative evidence that the pipeline can be reused across multiple cities without requiring separate processing implementations.

5.5 Data Processing Performance

Pipeline performance was evaluated using execution-time metrics collected during ingestion and Spark processing. Figure 5.4 shows the measured execution times.

The average total execution time was similar across Berlin, Bremen, and Frankfurt, with values of 20.55 s, 20.13 s, and 20.56 s, respectively. This indicates that the city-specific configurations had little effect on overall execution time under the evaluated workload.

For the Incident pipeline, execution time varied across processing stages. The Gold stage took 26.37 s, followed by Silver at 20.58 s, Bronze at 14.77 s, and API ingestion at 10.99 s. Thus, Gold processing was the main contributor to the observed execution time.

Overall, the pipeline showed consistent execution times across the evaluated cities, while the Incident pipeline showed greater variation between processing stages. These results reflect the tested workload and deployment environment and are not intended as a general scalability benchmark.

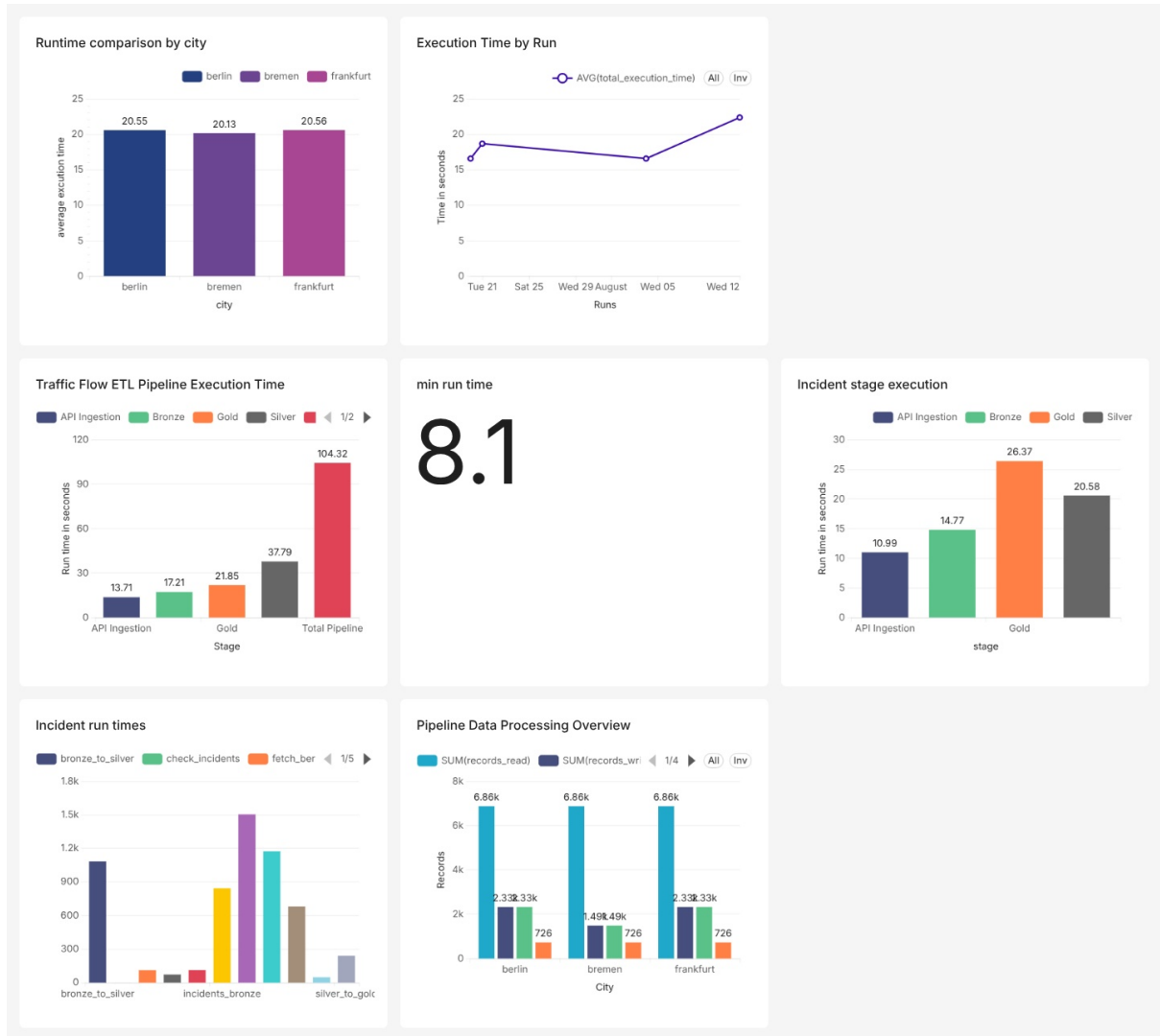


Figure 5.4: Performance overview of the pipeline

5.6 Dashboard and Alerting Evaluation

The Gold-layer output was loaded into PostgreSQL and used as the basis for the analytical dashboards implemented in Apache Superset. The dashboards provide an operational view of traffic conditions, incidents, and pipeline metrics.

The incident alerting component was evaluated by verifying that processed traffic incidents satisfying the configured alert conditions generated the corresponding email notification. This demonstrates the use of the processed data not only for analytical visualization but also for operational notification.

5.7 Discussion

The evaluation demonstrates that the proposed architecture addresses the three main design objectives.

First, reproducibility is supported at both the application and infrastructure levels. Docker Compose provides a consistent containerized runtime, while Terraform provides a reproducible infrastructure configuration.

Second, resilience is supported through Airflow orchestration, task dependencies, execution monitoring, and failure handling. The pipeline can therefore manage individual processing failures without requiring the complete environment to be manually reconstructed.

Third, reusability is demonstrated through the ability to process Berlin, Bremen, and Frankfurt using common processing components and standardized schemas. City-specific configuration is separated from the shared processing logic.

The evaluation also identifies practical limitations. The AWS deployment was used to evaluate infrastructure provisioning and reproducibility, while the complete ETL workload was evaluated in the local Docker environment. The limited resources of the selected cloud instance restricted its suitability for executing the complete multi-service workload simultaneously. Therefore, the AWS results should be interpreted primarily as an evaluation of deployment reproducibility rather than cloud execution performance.

5.8 Summary of Findings

Table 5.6: Summary of evaluation results

Research objective	Evaluation method	Finding
Reproducibility	Docker Compose and Terraform deployment	Environment can be recreated using version-controlled configuration.
Resilience	Airflow execution, retries, logs, and task states	Failures can be isolated and monitored at task level.
Reusability	Processing Berlin, Bremen, and Frankfurt	Common processing logic supports multiple cities.
Deployment efficiency	Manual versus Terraform provisioning	Estimated deployment effort reduced from 61 to 10.5 minutes.

Chapter 6

Conclusion and Future Work

6.1 Research Summary

This research presented the design, implementation, and evaluation of a reproducible, self-healing micro-batch ETL pipeline for smart-city traffic data analytics. The developed pipeline addressed key challenges in recurring traffic data processing, with a focus on reproducibility, resilience, and reusability.

The developed architecture enables automated collection, transformation, storage, and visualization of traffic flow and incident data obtained from the TomTom Traffic API. The system integrates workflow orchestration, distributed data processing, analytical storage, and visualization components into a modular data engineering framework.

The proposed solution applies configuration-driven processing to support multiple cities while maintaining a common processing workflow. The implemented architecture was evaluated using functional, performance, reproducibility, resilience, and reusability criteria to assess its effectiveness and suitability for smart city traffic data applications.

The evaluation results demonstrate that the developed pipeline successfully supports automated traffic data processing, structured analytical data delivery, workflow monitoring, and multi-city extensibility. The research shows that combining orchestration, distributed processing, standardized schemas, and reproducible deployment practices can improve the reliability and maintainability of smart city data engineering systems.

6.2 Research Contributions

This research's main contribution is the design and implementation of a reproducible smart-city traffic data engineering architecture that combines containerized services with automated infrastructure deployment. The research also developed a self-healing micro-batch ETL workflow using automated workflow orchestration, monitoring, logging, and recovery mechanisms to improve pipeline resilience. In addition, a reusable multi-city pro-

cessing approach was introduced through configuration-based geographic parameters and standardized schemas, enabling the same pipeline structure to process traffic data from different cities without requiring changes to the core processing logic. Finally, a comprehensive testing and evaluation approach was applied to verify the functional correctness of the implemented components and assess the system's performance, reproducibility, resilience, and reusability.

6.3 Research Questions Revisited

The following discussion summarizes how the proposed ETL pipeline addressed the research questions through its design, implementation, and evaluation.

RQ1: How can an ETL pipeline for smart-city traffic data achieve reproducible deployment and execution across different environments?

The results show that reproducibility can be supported by combining containerization and Infrastructure as Code. Docker Compose defines the application services and their dependencies in a reproducible runtime environment, while Terraform automates the provisioning of the infrastructure required to run the pipeline. The evaluation showed an estimated 82.8% reduction in deployment effort compared with the manual deployment approach.

RQ2: How can an ETL pipeline able to detect and recover from failures during recurring traffic data ingestion and processing with minimal manual intervention?

The implemented pipeline addresses this requirement through Airflow-based workflow orchestration, task monitoring, logging, automatic retries with backoff, and failure isolation. The separation of ingestion, processing, and alert workflows also limits failure propagation between pipeline components. These mechanisms allow transient failures to be recovered automatically while maintaining recurring data collection.

RQ3: How can an ETL pipeline support the ingestion and processing of traffic data across multiple cities?

The pipeline addresses this requirement through configuration-driven multi-city processing. City-specific parameters are separated from the common ingestion and transformation logic, allowing the same workflows to process Berlin, Bremen, and Frankfurt. A common processing structure and standardized output schema further support consistent analysis across the selected cities without requiring separate implementations for each location. Adding another city requires only a new configuration entry with its geographic bounding box and grid resolution; no changes to the ingestion, orchestration, or Spark transformation code are required.

6.4 Limitations

Although the proposed architecture successfully addressed the research objectives, several limitations remain.

First, the evaluation was conducted using three selected cities: Berlin, Bremen, and Frankfurt. While the configuration-based design supports extension to additional locations, large-scale validation involving a greater number of cities was outside the scope of this thesis.

Second, the implemented pipeline follows a micro-batch processing approach with scheduled execution intervals. A fully streaming architecture using technologies such as Apache Kafka or similar event-processing platforms was not investigated.

Third, the deployment experiments were influenced by available computational resources (Free tier AWS resource). The evaluation focused primarily on reproducibility and deployment consistency rather than large-scale cloud performance optimization.

Finally, this research focused on data engineering challenges rather than advanced traffic analytics. Prediction models, congestion forecasting, and machine learning-based traffic optimization were not included.

6.5 Future Work

Several directions can be explored to extend this research.

Future work could investigate the integration of real-time streaming technologies to replace micro-batch processing with continuous event-driven data processing. This would enable lower latency traffic monitoring and faster response to changing road conditions.

The architecture could also be extended with machine learning components for traffic prediction, congestion forecasting, and anomaly detection. Combining reliable data engineering pipelines with predictive analytics could provide additional value for smart city decision-support systems.

References

- [1] A. Caragliu, C. Del Bo, and P. Nijkamp, “Smart cities in europe,” *Journal of Urban Technology*, vol. 18, no. 2, pp. 65–82, 2011.
- [2] H. Chourabi, T. Nam, S. Walker, J. R. Gil-Garcia, S. Mellouli, K. Nahon, T. A. Pardo, and H. J. Scholl, “Understanding smart cities: An integrative framework,” in *2012 45th Hawaii International Conference on System Sciences*. IEEE, 2012, pp. 2289–2297.
- [3] S. A. Shaheen and M. A. Adkins, “Smart cities and intelligent transportation systems,” in *Smart Cities: Foundations, Principles, and Applications*. Wiley, 2017, pp. 1–24.
- [4] C. Goble, “Better scientific software for better scientific reproducibility,” *IEEE Computer*, vol. 47, no. 11, pp. 47–49, 2014.
- [5] S. E. Hareram, “Optimizing etl pipelines at scale: Lessons from pyspark and airflow integration,” *Journal of Information Systems Engineering and Management*, vol. 10, no. 60s, 2025.
- [6] D. Damaji, *Data Engineering with Python*. Packt, 2022.
- [7] J. Reis and M. Housley, *Fundamentals of Data Engineering: Plan and Build Robust Data Systems*. O’Reilly Media, 2022. [Online]. Available: <https://www.oreilly.com/library/view/fundamentals-of-data/9781098108298/>
- [8] M. Zaharia, “Apache spark: A unified engine for big data processing,” *Communications of the ACM*, 2016.
- [9] M. Fragkoulis, P. Carbone, V. Kalavri, and A. Katsifodimos, “A survey on the evolution of stream processing systems,” *The VLDB Journal*, vol. 33, pp. 507–541, 2024.
- [10] A. M. Garcia, D. Griebler, C. Schepke, and L. G. Fernandes, “Evaluating micro-batch and data frequency for stream processing applications on multi-cores,” in *2022 30th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*. IEEE, 2022, pp. 10–17.

- [11] R. Kimball and M. Ross, *The Data Warehouse Toolkit*. Wiley, 2013.
- [12] Databricks, “Medallion architecture: Improving data quality and reliability,” 2026. [Online]. Available: <https://www.databricks.com/glossary/medallion-architecture>
- [13] Apache Software Foundation, “Apache airflow documentation,” 2020. [Online]. Available: <https://airflow.apache.org/docs/>
- [14] C. Pahl, “Containerization and the paas cloud,” *IEEE Cloud Computing*, vol. 2, no. 3, pp. 24–31, 2015.
- [15] A. Rahman, R. Mahdavi-Hezaveh, and L. Williams, “Where are the gaps? a systematic mapping study of infrastructure as code research,” *arXiv preprint arXiv:1807.04872*, 2018.
- [16] X. Zeng, Y. Hui, J. Shen, A. Pavlo, W. McKinney, and H. Zhang, “An empirical evaluation of columnar storage formats,” *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 2733–2746, 2023.
- [17] M. Stonebraker and L. A. Rowe, “The design of postgres,” in *Proceedings of the 1986 ACM SIGMOD International Conference on Management of Data*. ACM, 1986, pp. 340–355.
- [18] D. Węclawski, R. Tomczyk, and P. Powroźnik, “Comparative analysis of selected data visualization methods,” *Journal of Computer Sciences Institute*, vol. 38, pp. 19–25, 2026.
- [19] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, “Internet of things for smart cities,” *IEEE Internet of Things Journal*, vol. 1, no. 1, pp. 22–32, 2014.
- [20] TomTom, “Tomtom traffic api documentation,” 2026. [Online]. Available: <https://developer.tomtom.com/traffic-api>
- [21] Apache Software Foundation, “Apache Superset Documentation,” 2026, [Online]. Available: <https://superset.apache.org/docs/> April 18, 2026.
- [22] HashiCorp. (2025) Terraform documentation. [Online]. Available: <https://developer.hashicorp.com/terraform/docs>
- [23] Docker Inc., “Docker documentation,” 2025. [Online]. Available: <https://docs.docker.com/>
- [24] B. Gebrewahid, “Smart city traffic etl pipeline,” 2026. [Online]. Available: <https://github.com/Birhanegb/smart-city-traffic-etl>